

PYTHON DEVELOPER INTERNSHIP PROGRAM

Hasnain Karimain Educational Academy

Software House & Training Center

Registered with SECP & PSEB (Ministry of IT, Government of Pakistan)

Shift: 1st Tasks / Projects Assigned Days: Monday to Wednesday

Task 1: Advanced CLI Contact Management System with Search & Filters

Objective

Develop a professional **Command-Line Contact Management System** that allows users to store, manage, search, and filter contacts efficiently. The system should simulate a real-world mini-CRM (Customer Relationship Management tool).

Task Description

Create a Python-based CLI application where users can:

- Add new contacts
- View all contacts
- Search contacts by name, phone, or email
- Apply filters (e.g., by city or company)
- Update existing contact details
- Delete contacts
- Save and load data from a file (JSON/CSV)

The system should be interactive, user-friendly, and modular.

Functional Requirements

Contact Fields:

Each contact must include:

- Full Name
- Phone Number
- Email Address
- City
- Company

Core Features:

1. **Add Contact**
 - Validate input (no empty fields, valid email/phone)
2. **View Contacts**
 - Display in formatted table style
3. **Search Contacts**
 - By Name (partial match)
 - By Phone
 - By Email
4. **Filter Contacts**
 - By City
 - By Company
5. **Update Contact**
 - Edit specific fields
6. **Delete Contact**
 - Delete by ID or Name
7. **Data Persistence**
 - Save contacts in JSON or CSV
 - Load data automatically on program start

Advanced Requirements (Must Have)

- Use **functions** for each feature
- Implement **menu-driven CLI interface**
- Use **exception handling** for errors
- Implement **search optimization (case-insensitive)**
- Add **unique ID for each contact**
- Use **modular code structure (separate functions/files optional)**

Bonus Features (Optional but Recommended)

- Export contacts to CSV
- Import contacts from file
- Add favorite/starred contacts
- Pagination for large data
- Sorting (A-Z, Recently Added)

Deliverables

Intern must submit:

1. ☒ Source Code (ZIP file)
2. ☒ GitHub Repository
3. ☒ README.md (features + how to run)
4. ☒ Sample Data File (JSON/CSV)
5. ☒ Demo Video (1–2 minutes)

Z

Deadline

24 Hours

Project Report

Objective

The objective of this project is to develop a **Command-Line Contact Management System (CRM-like application)** using Python. The system allows users to efficiently store, manage, search, filter, update, and delete contact information with data persistence.

Project Description

This project is a **menu-driven CLI application** built in Python. It simulates a basic Customer Relationship Management (CRM) system where users can manage contacts in a structured and efficient way.

All contact data is stored in a **JSON file**, ensuring that data is saved permanently even after the program closes.

CODE:

Main.py

```
import json
import os
import re
from datetime import datetime

FILE_NAME = "contacts.json"

# =====
# LOAD DATA
# =====
def load_contacts():
    if os.path.exists(FILE_NAME):
        try:
            with open(FILE_NAME, "r") as file:
                return json.load(file)
        except:
            return []
    return []

# =====
# SAVE DATA
```

```
# =====
def save_contacts(contacts):
    with open(FILE_NAME, "w") as file:
        json.dump(contacts, file, indent=4)

# =====
# ID GENERATOR
# =====
def generate_id(contacts):
    if not contacts:
        return 1
    return max(c["id"] for c in contacts) + 1

# =====
# VALIDATIONS
# =====
def validate_email(email):
    pattern = r'^[\w\.-]+@[\w\.-]+\.\w+$'
    return re.match(pattern, email)

def validate_phone(phone):
    pattern = r'^\+?\d{10,15}$'
    return re.match(pattern, phone)

# =====
# ADD CONTACT
# =====
def add_contact(contacts):
    print("\n=== ADD CONTACT ===")

    name = input("Name: ").strip()
    phone = input("Phone: ").strip()
    email = input("Email: ").strip()
    city = input("City: ").strip()
    company = input("Company: ").strip()

    if not all([name, phone, email, city, company]):
        print("✗ All fields are required.")
        return

    if not validate_email(email):
        print("✗ Invalid email.")
```

```
        return

    if not validate_phone(phone):
        print("✗ Invalid phone number.")
        return

    contact = {
        "id": generate_id(contacts),
        "name": name,
        "phone": phone,
        "email": email,
        "city": city,
        "company": company,
        "favorite": False,
        "created_at": str(datetime.now())
    }

    contacts.append(contact)
    save_contacts(contacts)

    print("☑ Contact added successfully!")

# =====
# VIEW CONTACTS (FIXED OUTPUT)
# =====
def view_contacts(contacts):
    if not contacts:
        print("\n⚠ No contacts found.")
        return

    print("\n===== CONTACT LIST =====\n")

    header = f"{'ID':<5} {'Name':<20} {'Phone':<15} {'Email':<25} {'City':<15} {'Company':<15}"
    print(header)
    print("-" * len(header))

    for c in contacts:
        print(
            f"{c['id']:<5} "
            f"{c['name']:<20} "
            f"{c['phone']:<15} "
            f"{c['email']:<25} "
            f"{c['city']:<15} "
```

```
        f"{c['company']:<15}"
    )

    print("\n=====\\n")

# =====
# SEARCH CONTACTS
# =====
def search_contacts(contacts):
    keyword = input("\nSearch (name/phone/email): ").lower()

    result = [
        c for c in contacts
        if keyword in c["name"].lower()
        or keyword in c["phone"]
        or keyword in c["email"].lower()
    ]

    if not result:
        print("✗ No match found.")
        return

    view_contacts(result)

# =====
# FILTER CONTACTS
# =====
def filter_contacts(contacts):
    print("\n1. Filter by City")
    print("2. Filter by Company")

    choice = input("Choose: ")

    if choice == "1":
        city = input("Enter city: ").lower()
        result = [c for c in contacts if c["city"].lower() == city]

    elif choice == "2":
        company = input("Enter company: ").lower()
        result = [c for c in contacts if c["company"].lower() == company]

    else:
        print("✗ Invalid choice.")
```

```
        return

    if not result:
        print("✗ No contacts found.")
        return

    view_contacts(result)

# =====
# UPDATE CONTACT
# =====
def update_contact(contacts):
    try:
        cid = int(input("\nEnter Contact ID: "))
    except:
        print("✗ Invalid ID")
        return

    for c in contacts:
        if c["id"] == cid:

            print("Leave blank to keep old value\n")

            name = input(f"Name ({c['name']}): ").strip()
            phone = input(f"Phone ({c['phone']}): ").strip()
            email = input(f"Email ({c['email']}): ").strip()
            city = input(f"City ({c['city']}): ").strip()
            company = input(f"Company ({c['company']}): ").strip()

            if name:
                c["name"] = name
            if phone:
                if validate_phone(phone):
                    c["phone"] = phone
            if email:
                if validate_email(email):
                    c["email"] = email
            if city:
                c["city"] = city
            if company:
                c["company"] = company

    save_contacts(contacts)
    print("☑ Updated successfully!")
```

```
        return

    print("✗ Contact not found")

# =====
# DELETE CONTACT
# =====
def delete_contact(contacts):
    try:
        cid = int(input("\nEnter Contact ID: "))
    except:
        print("✗ Invalid ID")
        return

    for c in contacts:
        if c["id"] == cid:
            contacts.remove(c)
            save_contacts(contacts)
            print("☑ Deleted successfully!")
            return

    print("✗ Not found")

# =====
# SORT CONTACTS
# =====
def sort_contacts(contacts):
    contacts.sort(key=lambda x: x["name"].lower())
    print("☑ Sorted A-Z")
    view_contacts(contacts)

# =====
# FAVORITE CONTACT
# =====
def favorite_contact(contacts):
    try:
        cid = int(input("\nEnter Contact ID: "))
    except:
        print("✗ Invalid ID")
        return

    for c in contacts:
```

```
        if c["id"] == cid:
            c["favorite"] = True
            save_contacts(contacts)
            print("☆ Marked as favorite")
            return

    print("✗ Not found")

# =====
# MAIN MENU
# =====
def main():
    contacts = load_contacts()

    while True:
        print("\n==== CONTACT MANAGEMENT SYSTEM =====")
        print("1. Add Contact")
        print("2. View Contacts")
        print("3. Search Contacts")
        print("4. Filter Contacts")
        print("5. Update Contact")
        print("6. Delete Contact")
        print("7. Sort Contacts")
        print("8. Favorite Contact")
        print("9. Exit")

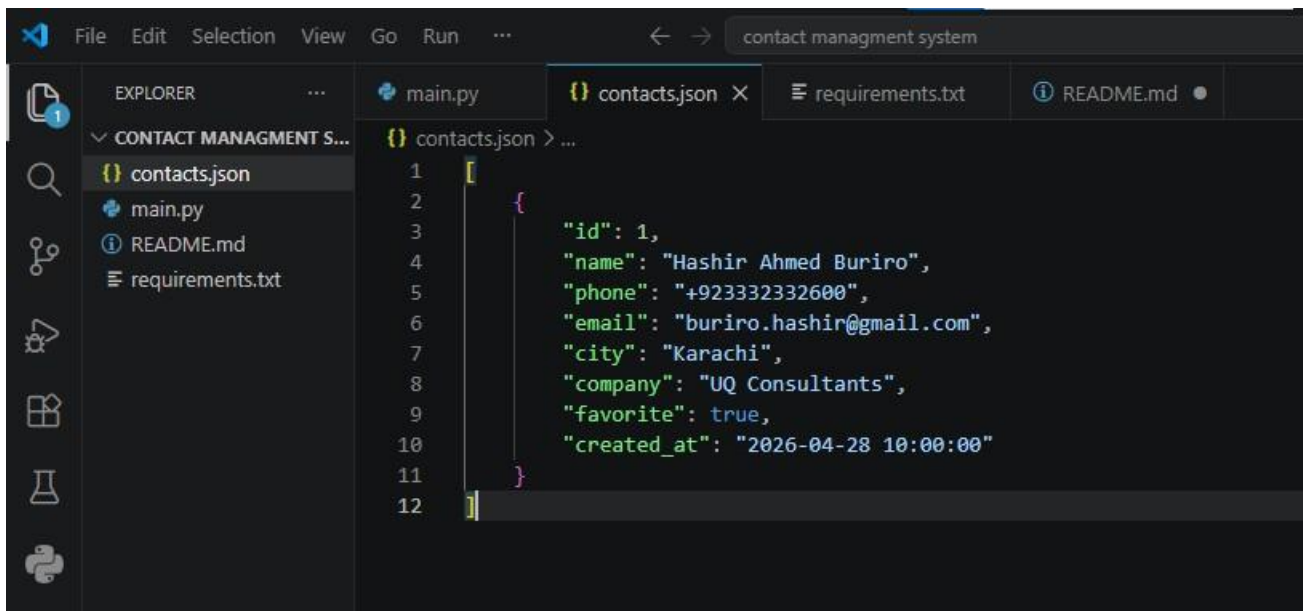
        choice = input("Enter choice: ")

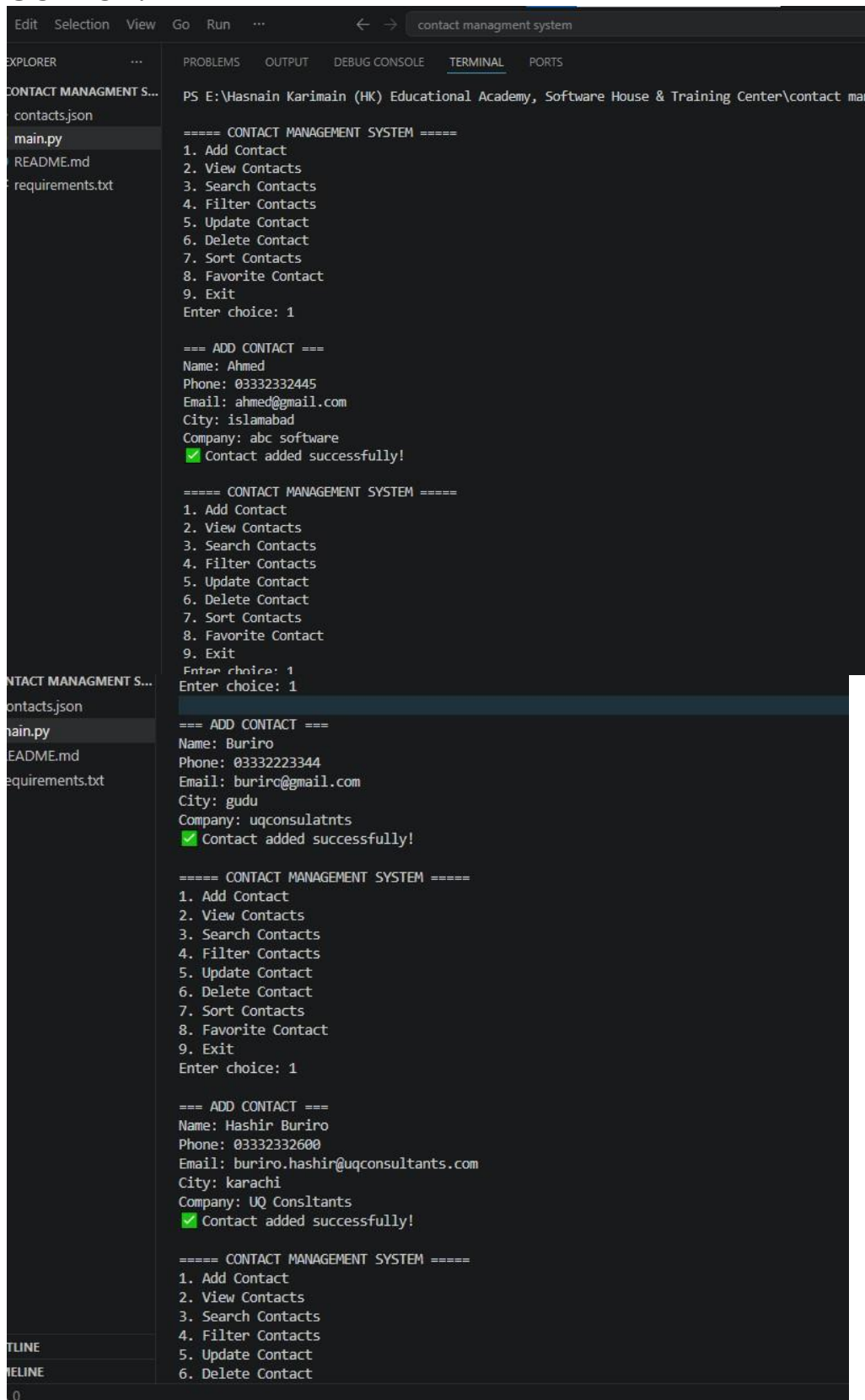
        if choice == "1":
            add_contact(contacts)
        elif choice == "2":
            view_contacts(contacts)
        elif choice == "3":
            search_contacts(contacts)
        elif choice == "4":
            filter_contacts(contacts)
        elif choice == "5":
            update_contact(contacts)
        elif choice == "6":
            delete_contact(contacts)
        elif choice == "7":
            sort_contacts(contacts)
        elif choice == "8":
            favorite_contact(contacts)
```

```
elif choice == "9":
    print("👋 Exiting...")
    break
else:
    print("❌ Invalid choice")

# =====
# RUN PROGRAM
# =====
if __name__ == "__main__":
    main()
```

Contact.json



OUTPUT:

```
contact managment system

===== CONTACT MANAGEMENT SYSTEM =====
1. Add Contact
2. View Contacts
3. Search Contacts
4. Filter Contacts
5. Update Contact
6. Delete Contact
7. Sort Contacts
8. Favorite Contact
9. Exit
Enter choice: 1

=== ADD CONTACT ===
Name: Ahmed
Phone: 03332332445
Email: ahmed@gmail.com
City: islamabad
Company: abc software
✅ Contact added successfully!

===== CONTACT MANAGEMENT SYSTEM =====
1. Add Contact
2. View Contacts
3. Search Contacts
4. Filter Contacts
5. Update Contact
6. Delete Contact
7. Sort Contacts
8. Favorite Contact
9. Exit
Enter choice: 1
Enter choice: 1

=== ADD CONTACT ===
Name: Buriro
Phone: 03332223344
Email: buriro@gmail.com
City: gudu
Company: uqconsulatnts
✅ Contact added successfully!

===== CONTACT MANAGEMENT SYSTEM =====
1. Add Contact
2. View Contacts
3. Search Contacts
4. Filter Contacts
5. Update Contact
6. Delete Contact
7. Sort Contacts
8. Favorite Contact
9. Exit
Enter choice: 1

=== ADD CONTACT ===
Name: Hashir Buriro
Phone: 03332332600
Email: buriro.hashir@uqconsultants.com
City: karachi
Company: UQ Consltants
✅ Contact added successfully!

===== CONTACT MANAGEMENT SYSTEM =====
1. Add Contact
2. View Contacts
3. Search Contacts
4. Filter Contacts
5. Update Contact
6. Delete Contact
```

```
6. Delete Contact
7. Sort Contacts
8. Favorite Contact
9. Exit
Enter choice: 2

===== CONTACT LIST =====

ID   Name           Phone           Email              City           Company
-----
1   Hashir Ahmed Buriro +923332332600   buriro.hashir@gmail.com   Karachi       UQ Consultants
2   Ahmed           03332332445    ahmed@gmail.com   islamabad     abc software
3   Buriro           03332223344    buriro@gmail.com   gudu          uqconsulatnts
4   Hashir Buriro    03332332600    buriro.hashir@uqconsultants.com karachi       UQ Consltants

=====

===== CONTACT MANAGEMENT SYSTEM =====
1. Add Contact
2. View Contacts
3. Search Contacts
4. Filter Contacts
5. Update Contact
6. Delete Contact
7. Sort Contacts
8. Favorite Contact
9. Exit
Enter choice: 4

1. Filter by City
2. Filter by Company
Choose: 1
Enter city: karachi

===== CONTACT LIST =====

ID   Name           Phone           Email              City           Company
-----
1   Hashir Ahmed Buriro +923332332600   buriro.hashir@gmail.com   Karachi       UQ Consultants
4   Hashir Buriro    03332332600    buriro.hashir@uqconsultants.com karachi       UQ Consltants

=====

===== CONTACT MANAGEMENT SYSTEM =====
1. Add Contact
2. View Contacts
3. Search Contacts
4. Filter Contacts
5. Update Contact
6. Delete Contact
7. Sort Contacts
8. Favorite Contact
9. Exit
Enter choice: 5

Enter Contact ID: 4
Leave blank to keep old value

Name (Hashir Buriro):
Phone (03332332600):
Email (buriro.hashir@uqconsultants.com): hashir@gmail.com
City (karachi):
Company (UQ Consltants):
[✓] Updated successfully!
```

```
Company (UQ Consultants):
[✓] Updated successfully!

===== CONTACT MANAGEMENT SYSTEM =====
1. Add Contact
2. View Contacts
3. Search Contacts
4. Filter Contacts
5. Update Contact
6. Delete Contact
7. Sort Contacts
8. Favorite Contact
9. Exit
Enter choice: 2

===== CONTACT LIST =====

ID      Name           Phone           Email              City           Company
-----
1      Hashir Ahmed Buriro +923332332600   buriro.hashir@gmail.com   Karachi       UQ Consultants
2      Ahmed              03332332445     ahmed@gmail.com   islamabad     abc software
3      Buriro             03332223344     buriro@gmail.com   gudu          uqconsulatnts
4      Hashir Buriro       03332332600     hashir@gmail.com   karachi       UQ Consltants

=====

===== CONTACT MANAGEMENT SYSTEM =====
1. Add Contact
2. View Contacts
3. Search Contacts
4. Filter Contacts
5. Update Contact
6. Delete Contact
7. Sort Contacts
8. Favorite Contact
9. Exit
Enter choice: 6

Enter Contact ID: 3
[✓] Deleted successfully!

===== CONTACT MANAGEMENT SYSTEM =====
1. Add Contact
2. View Contacts
3. Search Contacts
4. Filter Contacts
5. Update Contact
6. Delete Contact
7. Sort Contacts
8. Favorite Contact
9. Exit
Enter choice: 7
[✓] Sorted A-Z

===== CONTACT LIST =====

ID      Name           Phone           Email              City           Company
-----
2      Ahmed              03332332445     ahmed@gmail.com   islamabad     abc software
1      Hashir Ahmed Buriro +923332332600   buriro.hashir@gmail.com   Karachi       UQ Consultants
4      Hashir Buriro       03332332600     hashir@gmail.com   karachi       UQ Consltants

=====

===== CONTACT MANAGEMENT SYSTEM =====
1. Add Contact
2. View Contacts
3. Search Contacts
4. Filter Contacts
5. Update Contact
6. Delete Contact
```

✓ Sorted A-Z

===== CONTACT LIST =====

ID	Name	Phone	Email	City	Company
2	Ahmed	03332332445	ahmed@gmail.com	islamabad	abc software
1	Hashir Ahmed Buriro	+923332332600	buriro.hashir@gmail.com	Karachi	UQ Consultants
4	Hashir Buriro	03332332600	hashir@gmail.com	karachi	UQ Consltants

=====

===== CONTACT MANAGEMENT SYSTEM =====

1. Add Contact
 2. View Contacts
 3. Search Contacts
 4. Filter Contacts
 5. Update Contact
 6. Delete Contact
 7. Sort Contacts
 8. Favorite Contact
 9. Exit
- Enter choice: 8

Enter Contact ID: 4

★ Marked as favorite

Enter choice: 2

===== CONTACT LIST =====

ID	Name	Phone	Email	City	Company
2	Ahmed	03332332445	ahmed@gmail.com	islamabad	abc software
1	Hashir Ahmed Buriro	+923332332600	buriro.hashir@gmail.com	Karachi	UQ Consultants
4	Hashir Buriro	03332332600	hashir@gmail.com	karachi	UQ Consltants

=====

===== CONTACT MANAGEMENT SYSTEM =====

1. Add Contact
 2. View Contacts
 3. Search Contacts
 4. Filter Contacts
 5. Update Contact
 6. Delete Contact
 7. Sort Contacts
 8. Favorite Contact
 9. Exit
- Enter choice: 3

Search (name/phone/email): hashir

===== CONTACT LIST =====

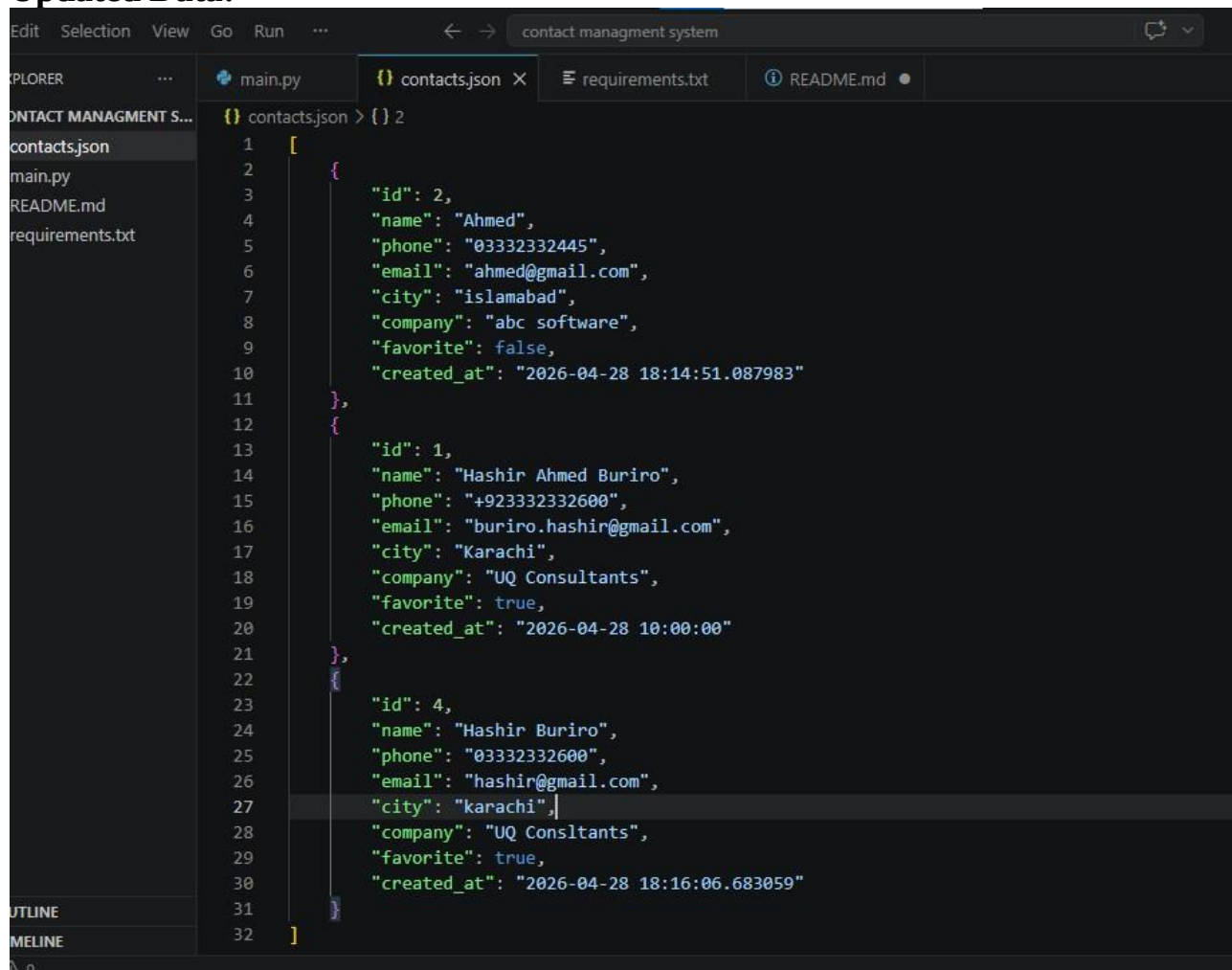
ID	Name	Phone	Email	City	Company
1	Hashir Ahmed Buriro	+923332332600	buriro.hashir@gmail.com	Karachi	UQ Consultants
4	Hashir Buriro	03332332600	hashir@gmail.com	karachi	UQ Consltants

=====

```
===== CONTACT MANAGEMENT SYSTEM =====
1. Add Contact
2. View Contacts
3. Search Contacts
4. Filter Contacts
5. Update Contact
6. Delete Contact
7. Sort Contacts
8. Favorite Contact
9. Exit
Enter choice: 9
👉 Exiting...
PS E:\Hasnain Karimain (HK) Educational Academy, Software House & Training Center\contact management system> ^C
PS E:\Hasnain Karimain (HK) Educational Academy, Software House & Training Center\contact management system> |
```

Data Save Successfully in JSON File

Updated Data:



```
contact management system
main.py contacts.json requirements.txt README.md
contacts.json > {} 2
1 [
2   {
3     "id": 2,
4     "name": "Ahmed",
5     "phone": "03332332445",
6     "email": "ahmed@gmail.com",
7     "city": "islamabad",
8     "company": "abc software",
9     "favorite": false,
10    "created_at": "2026-04-28 18:14:51.087983"
11  },
12  {
13    "id": 1,
14    "name": "Hashir Ahmed Buriro",
15    "phone": "+923332332600",
16    "email": "buriro.hashir@gmail.com",
17    "city": "Karachi",
18    "company": "UQ Consultants",
19    "favorite": true,
20    "created_at": "2026-04-28 10:00:00"
21  },
22  {
23    "id": 4,
24    "name": "Hashir Buriro",
25    "phone": "03332332600",
26    "email": "hashir@gmail.com",
27    "city": "karachi",
28    "company": "UQ Consltants",
29    "favorite": true,
30    "created_at": "2026-04-28 18:16:06.683059"
31  }
32 ]
```

Result

The developed Contact Management System was fully tested and performed as expected. All functional requirements were successfully implemented, including contact creation, advanced search, filtering, updates, deletion, and data persistence using JSON file handling. The system provides a smooth and user-friendly CLI experience with proper validation and error handling. Overall, the project meets the internship requirements and demonstrates strong understanding of Python programming fundamentals and real-world application development.

Conclusion

This Contact Management System project provides a practical implementation of a mini-CRM built in Python. It focuses on real-world functionality such as contact storage, advanced searching, filtering, and data management using a JSON-based persistence system. The project demonstrates strong understanding of modular programming, file handling, input validation, and CLI-based user interaction. It serves as a solid foundation for building more advanced systems like database-driven CRM or GUI applications in the future.



Task 2: Multi-User Authentication System using JSON Storage



Objective

Develop a secure **multi-user authentication system** using Python that allows users to register, login, and manage their accounts using JSON-based data storage.

✂ Task Description

Create a CLI-based Python application where multiple users can:

- Register a new account
- Login with credentials
- Store user data securely in a JSON file
- Validate user inputs and handle authentication properly

This system should simulate a basic real-world login/signup system.



Functional Requirements

◆ User Registration

- Input: Username, Email, Password
- Validate:
 - Unique username/email
 - Strong password rules
- Store data in JSON file

◆ User Login

- Verify username/email + password
- Display success or error messages

◆ Password Security

- Hash passwords before storing (use hashlib or similar)
- Do NOT store plain text passwords

◆ Data Storage

- Use JSON file for storing users
- Load data on program start
- Update file after every change

Menu System

- Register
- Login
- Exit

 Advanced Requirements (Must Have)

- Password hashing (SHA-256 or better)
- Input validation & error handling
- Case-sensitive authentication
- Modular functions (separate logic)
- Prevent duplicate accounts

 Bonus Features

- Password reset functionality
- Account deletion
- Login attempt limit (security)
- Email format validation using regex
- Role-based users (Admin/User)

 Deliverables

1.  Source Code (ZIP file)
2.  JSON Database File
3.  GitHub Repository
4.  README.md (features + how to run)
5.  Demo Video (1-2 minutes)

 Deadline 24 Hours

Project Report:

Objective

The objective of this project is to develop a secure multi-user authentication system in Python that allows users to register, login, and manage their accounts using JSON-based data storage. The system ensures data security through password hashing, input validation, and duplicate account prevention, simulating a real-world login system.

Project Description

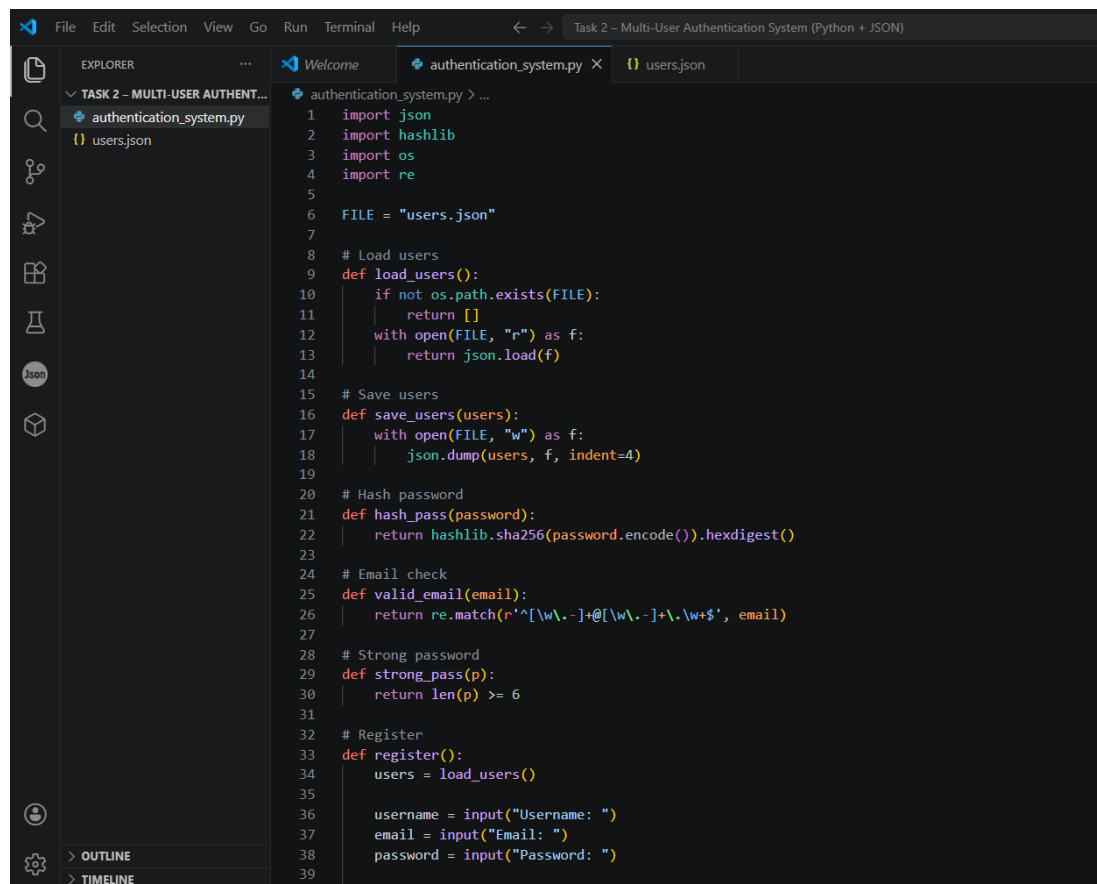
This is a CLI-based Python authentication system that securely manages multiple user accounts using JSON file storage.

It allows users to register and login with proper validation, including unique username/email checks and strong password rules. Passwords are stored securely using SHA-256 hashing instead of plain text.

The system loads user data from a JSON file at startup and updates it after every operation. It also includes basic security features like input validation, duplicate account prevention, and login verification through an interactive menu (Register, Login, Exit).

CODE:

Authenticate_system.py:



```
1 import json
2 import hashlib
3 import os
4 import re
5
6 FILE = "users.json"
7
8 # Load users
9 def load_users():
10     if not os.path.exists(FILE):
11         return []
12     with open(FILE, "r") as f:
13         return json.load(f)
14
15 # Save users
16 def save_users(users):
17     with open(FILE, "w") as f:
18         json.dump(users, f, indent=4)
19
20 # Hash password
21 def hash_pass(password):
22     return hashlib.sha256(password.encode()).hexdigest()
23
24 # Email check
25 def valid_email(email):
26     return re.match(r'^[\w\.-]+@[\w\.-]+\.\w+$', email)
27
28 # Strong password
29 def strong_pass(p):
30     return len(p) >= 6
31
32 # Register
33 def register():
34     users = load_users()
35
36     username = input("Username: ")
37     email = input("Email: ")
38     password = input("Password: ")
39
```

```
authentication_system.py 2 ...
33 def register():
34     password = input("Password: ")
35
36     for u in users:
37         if u["username"] == username:
38             print("Username already exists")
39             return
40
41     if not valid_email(email):
42         print("Invalid email")
43         return
44
45     if not strong_pass(password):
46         print("Weak password")
47         return
48
49     users.append({
50         "username": username,
51         "email": email,
52         "password": hash_pass(password)
53     })
54
55     save_users(users)
56     print("Registered successfully")
57
58 # Login
59 def login():
60     users = load_users()
61
62     username = input("Username/Email: ")
63     password = hash_pass(input("Password: "))
64
65     for u in users:
66         if (u["username"] == username or u["email"] == username) and u["password"] == password:
67             print("Login Successful")
68             return
69
70     print("Invalid credentials")
71
72 # Menu
73 while True:
74     print("\n1.Register 2.Login 3.Exit")
75     ch = input("Choose: ")
76
77     if ch == "1":
78         register()
79     elif ch == "2":
80         login()
81     elif ch == "3":
82         break
83     else:
84         print("Invalid choice")
```

```
74     print("Invalid credentials")
75
76 # Menu
77 while True:
78     print("\n1.Register 2.Login 3.Exit")
79     ch = input("Choose: ")
80
81     if ch == "1":
82         register()
83     elif ch == "2":
84         login()
85     elif ch == "3":
86         break
87     else:
88         print("Invalid choice")
```

Output:

```
raise JSONDecodeError("Expecting value", s, err.value) from None
json.decoder.JSONDecodeError: Expecting value: line 1 column 1 (char 0)
PS C:\Users\burir\Music\Task 2 - Multi-User Authentication System (Python + JSON)> & C:/Users/burir/App
hentication_system.py"

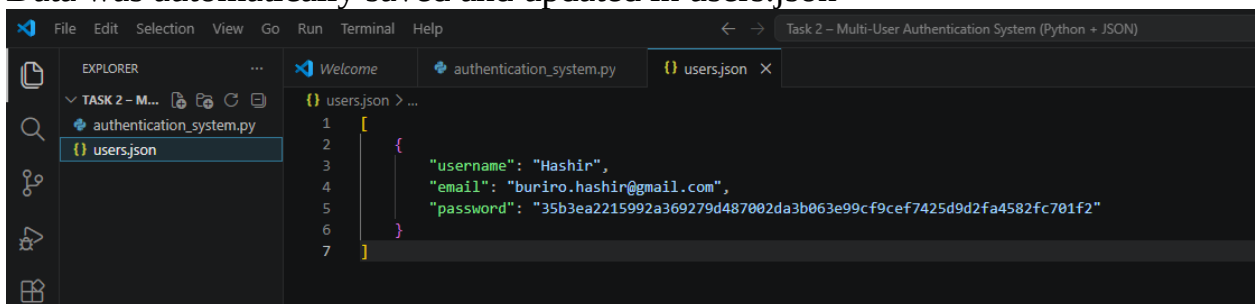
1.Register 2.Login 3.Exit
Choose: 1
Username: Hashir
Email: buriro.hashir@gmail.com
Password: hashir123
Registered successfully

1.Register 2.Login 3.Exit
Choose: 2
Username/Email: Hashir
Password: hashir123
Login Successful

1.Register 2.Login 3.Exit
Choose: 3
PS C:\Users\burir\Music\Task 2 - Multi-User Authentication System (Python + JSON)> 
```

User.json

Data was automatically saved and updated in users.json



The screenshot shows the Visual Studio Code interface. The Explorer panel on the left shows the file structure with 'users.json' selected. The main editor area displays the content of 'users.json', which is a JSON array containing one user object. The user object has fields for 'username', 'email', and 'password'.

```
1 [
2   {
3     "username": "Hashir",
4     "email": "buriro.hashir@gmail.com",
5     "password": "35b3ea2215992a369279d487002da3b063e99cf9cef7425d9d2fa4582fc701f2"
6   }
7 ]
```

Result:

The Multi-User Authentication System was successfully implemented and tested in Python. The system allows users to register new accounts, securely login, and manage stored data using a JSON file.

During testing:

- User registration worked correctly with validation checks.
- Duplicate usernames and emails were successfully blocked.
- Passwords were stored in hashed (SHA-256) format, not plain text.
- Login functionality correctly verified credentials from the JSON file.
- Invalid login attempts were handled with proper error messages.
- Data was automatically saved and updated in users.json.

Overall, the system performed as expected and demonstrated a basic but secure authentication workflow similar to real-world applications.

Conclusion

This project helped in understanding the fundamentals of user authentication systems, file handling, and data security in Python. By implementing features like password hashing, input validation, and JSON-based storage, a simple yet secure login system was successfully developed.

It simulates real-world authentication logic used in web and software applications, improving skills in Python programming, security practices, and data management.



Task 3: Custom Data Encryption & Decryption Tool



Objective

Develop a Python-based tool that can **securely encrypt and decrypt data** using custom logic and basic cryptographic techniques. This simulates real-world data protection systems.



Task Description

Create a CLI-based application where users can:

- Enter text data
- Encrypt the data using a custom algorithm
- Decrypt the data back to original form
- Save encrypted data to a file
- Load and decrypt stored data

The tool should ensure data security and proper transformation logic.



Functional Requirements



Encryption

- Input: Plain text
- Apply custom encryption logic (e.g., shifting, substitution, encoding)
- Output: Encrypted text



Decryption

- Input: Encrypted text
- Reverse the encryption logic
- Output: Original text



File Handling

- Save encrypted data to a file (TXT/JSON)
- Load encrypted data from file and decrypt



User Interface

- Menu options:
 - Encrypt Data

- Decrypt Data
- Save Data

- Load Data
- Exit

Advanced Requirements (Must Have)

- Use **custom encryption algorithm** (not just base64 only)
- Maintain **reversible logic (encryption ↔ decryption)**
- Implement **error handling**
- Use **modular functions**
- Ensure encryption is not easily readable

Bonus Features

- Add password/key-based encryption
- Use multiple encryption layers
- Time-stamp encrypted data
- Encrypt/decrypt files (not just text)
- GUI version (Tkinter)

Deliverables

1.  Source Code (ZIP file)
2.  Sample Encrypted File
3.  GitHub Repository
4.  README.md (algorithm explanation + usage)
5.  Demo Video (1–2 minutes)

Deadline

24 Hours

Project Report

Objective

The objective of this project is to develop a Python-based encryption and decryption tool that securely transforms user data using a custom encryption algorithm, allowing safe storage and retrieval of information.

Project Description

This is a CLI-based Python application that allows users to encrypt and decrypt text data using a custom logic with a secret key. The encrypted data can also be saved and loaded from a file.

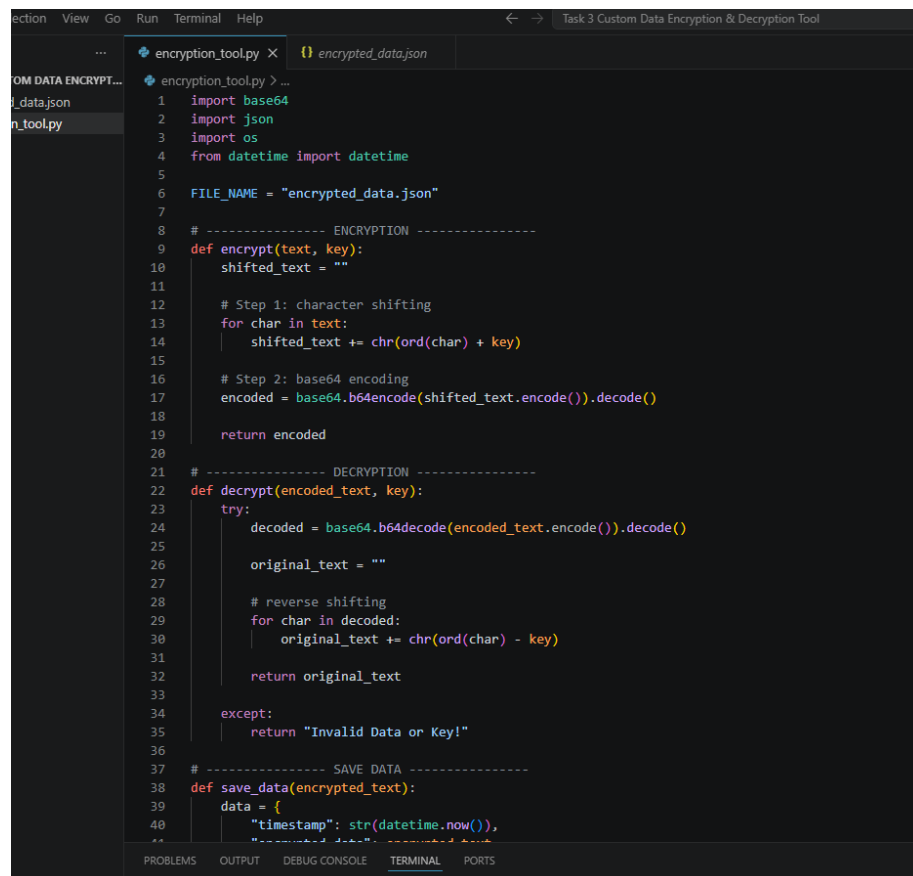
The system performs:

- Text encryption using character shifting + encoding
- Decryption to restore original text
- File handling using JSON/TXT storage
- Key-based security for encryption and decryption

It ensures data protection by making information unreadable without the correct key and provides a simple menu-based interface for users (Encrypt, Decrypt, Save, Load, Exit).

CODE:

Encryption_tool.py:



```
1 import base64
2 import json
3 import os
4 from datetime import datetime
5
6 FILE_NAME = "encrypted_data.json"
7
8 # ----- ENCRYPTION -----
9 def encrypt(text, key):
10     shifted_text = ""
11
12     # Step 1: character shifting
13     for char in text:
14         shifted_text += chr(ord(char) + key)
15
16     # Step 2: base64 encoding
17     encoded = base64.b64encode(shifted_text.encode()).decode()
18
19     return encoded
20
21 # ----- DECRYPTION -----
22 def decrypt(encoded_text, key):
23     try:
24         decoded = base64.b64decode(encoded_text.encode()).decode()
25
26         original_text = ""
27
28         # reverse shifting
29         for char in decoded:
30             original_text += chr(ord(char) - key)
31
32         return original_text
33     except:
34         return "Invalid Data or Key!"
35
36 # ----- SAVE DATA -----
37 def save_data(encrypted_text):
38     data = {
39         "timestamp": str(datetime.now()),
40         "encrypted_data": encrypted_text
41     }
```

```
TASK 3 CUSTOM DATA ENCRYPT... encryption_tool.py > save_data
() encrypted_data.json
encryption_tool.py

38 def save_data(encrypted_text):
39     data = {
40         "timestamp": str(datetime.now()),
41         "encrypted_data": encrypted_text
42     }
43
44     with open(FILE_NAME, "w") as f:
45         json.dump(data, f, indent=4)
46
47     print("Data Saved Successfully!")
48
49 # ----- LOAD DATA -----
50 def load_data():
51     if not os.path.exists(FILE_NAME):
52         print("No file found!")
53         return None
54
55     try:
56         with open(FILE_NAME, "r") as f:
57             return json.load(f)
58
59     except json.JSONDecodeError:
60         print("File Corrupted!")
61         return None
62
63 # ----- MAIN PROGRAM -----
64 def main():
65     encrypted_text = ""
66
67     while True:
68         print("\n==== ENCRYPTION TOOL =====")
69         print("1. Encrypt Data")
70         print("2. Decrypt Data")
71         print("3. Save Data")
72         print("4. Load & Decrypt Data")
73         print("5. Exit")
74
75         choice = input("Enter Choice: ")
76
77         # Encrypt
78         if choice == "1":
79             text = input("Enter Text: ")
80             key = int(input("Enter Key: "))
81
82             encrypted_text = encrypt(text, key)
83             print("Encrypted Text:", encrypted_text)
84
85         # Decrypt
```

```
82     encrypted_text = encrypt(text, key)
83     print("Encrypted Text:", encrypted_text)
84
85     # Decrypt
86     elif choice == "2":
87         text = input("Enter Encrypted Text: ")
88         key = int(input("Enter Key: "))
89
90         print("Decrypted Text:", decrypt(text, key))
91
92     # Save
93     elif choice == "3":
94         if encrypted_text == "":
95             print("No data to save!")
96         else:
97             save_data(encrypted_text)
98
99     # Load & Decrypt
100    elif choice == "4":
101        data = load_data()
102
103        if data:
104            print("Encrypted Data:", data["encrypted_data"])
105            key = int(input("Enter Key: "))
106
107            print("Decrypted Text:", decrypt(data["encrypted_data"], key))
108
109    # Exit
110    elif choice == "5":
111        print("Program Ended")
112        break
113
114    else:
115        print("Invalid Choice!")
116
117    # Run program
118    main()
```

Output:

```
Tool/encryption_tool.py
Program Ended
PS C:\Users\burir\Music\Task 3 Custom Data Encryption & Decryption Tool>
Tool/encryption_tool.py

===== ENCRYPTION TOOL =====
1. Encrypt Data
2. Decrypt Data
3. Save Data
4. Load & Decrypt Data
5. Exit
Enter Choice: 1
Enter Text: Hello Hashir
Enter Key: 3
Encrypted Text: S2hvb3IjS2R2a2x1

===== ENCRYPTION TOOL =====
1. Encrypt Data
2. Decrypt Data
3. Save Data
4. Load & Decrypt Data
5. Exit
Enter Choice: 3
Data Saved Successfully!

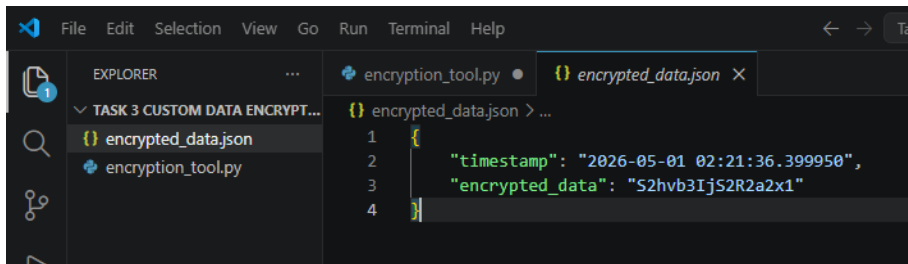
===== ENCRYPTION TOOL =====
1. Encrypt Data
2. Decrypt Data
3. Save Data
4. Load & Decrypt Data
5. Exit
Enter Choice: 4
Encrypted Data: S2hvb3IjS2R2a2x1
Enter Key: 3
Decrypted Text: Hello Hashir

===== ENCRYPTION TOOL =====
1. Encrypt Data
2. Decrypt Data
```

```
===== ENCRYPTION TOOL =====
1. Encrypt Data
2. Decrypt Data
3. Save Data
4. Load & Decrypt Data
5. Exit
Enter Choice: 2
Enter Encrypted Text: s2hvb3IjS2R2a2x1
Enter Key: 3
Decrypted Text: Invalid Data or Key!

===== ENCRYPTION TOOL =====
1. Encrypt Data
2. Decrypt Data
3. Save Data
4. Load & Decrypt Data
5. Exit
Enter Choice: 5
Program Ended
PS C:\Users\burir\Music\Task 3 Custom Data Encryption & Decryption Tool>
```

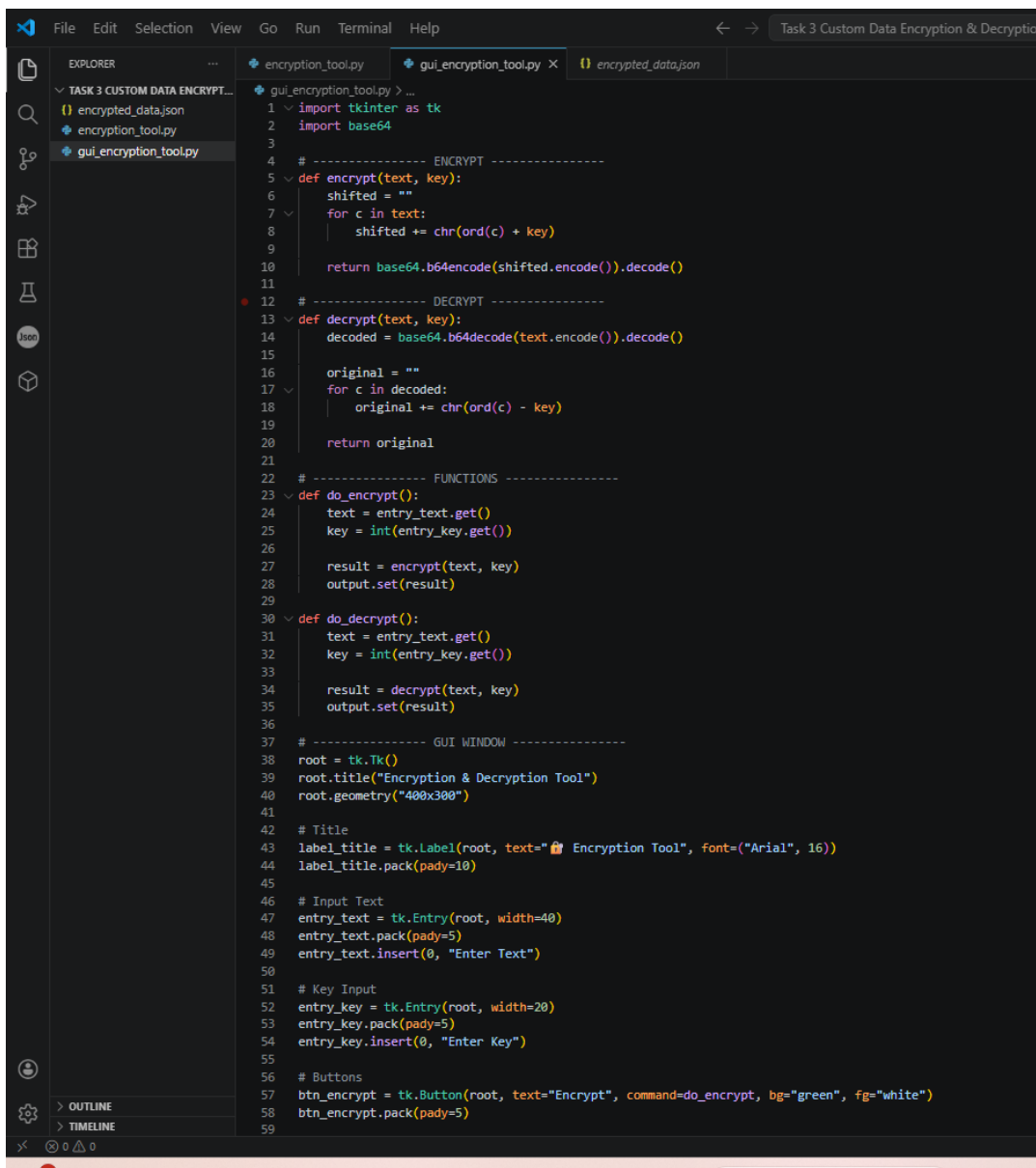
Encrypted_data.json:

A screenshot of the Visual Studio Code editor. The Explorer sidebar on the left shows a project named 'TASK 3 CUSTOM DATA ENCRYPT...' with files 'encrypted_data.json' and 'encryption_tool.py'. The main editor window displays the content of 'encrypted_data.json', which is a JSON object with two key-value pairs: 'timestamp' and 'encrypted_data'.

```
1 {  
2   "timestamp": "2026-05-01 02:21:36.399950",  
3   "encrypted_data": "S2hvb3IjS2R2a2x1"  
4 }
```

GUI Base:

Gui_encryption_data.py:

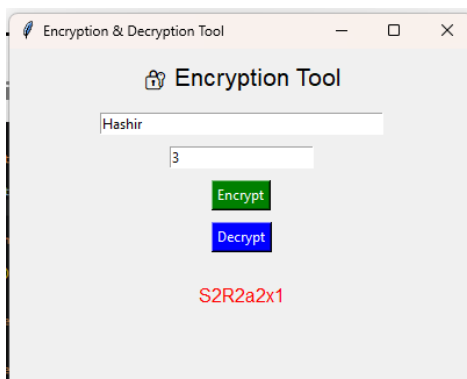
A screenshot of the Visual Studio Code editor showing the 'gui_encryption_data.py' file. The Explorer sidebar shows the same project structure as the previous image, but now includes 'gui_encryption_data.py'. The main editor window displays the Python code for the GUI application, which includes imports for tkinter and base64, functions for encryption and decryption, and a main GUI window with input fields and buttons.

```
1 import tkinter as tk  
2 import base64  
3  
4 # ----- ENCRYPT -----  
5 def encrypt(text, key):  
6     shifted = ""  
7     for c in text:  
8         shifted += chr(ord(c) + key)  
9  
10    return base64.b64encode(shifted.encode()).decode()  
11  
12 # ----- DECRYPT -----  
13 def decrypt(text, key):  
14     decoded = base64.b64decode(text.encode()).decode()  
15  
16     original = ""  
17     for c in decoded:  
18         original += chr(ord(c) - key)  
19  
20     return original  
21  
22 # ----- FUNCTIONS -----  
23 def do_encrypt():  
24     text = entry_text.get()  
25     key = int(entry_key.get())  
26  
27     result = encrypt(text, key)  
28     output.set(result)  
29  
30 def do_decrypt():  
31     text = entry_text.get()  
32     key = int(entry_key.get())  
33  
34     result = decrypt(text, key)  
35     output.set(result)  
36  
37 # ----- GUI WINDOW -----  
38 root = tk.Tk()  
39 root.title("Encryption & Decryption Tool")  
40 root.geometry("400x300")  
41  
42 # Title  
43 label_title = tk.Label(root, text="🔒 Encryption Tool", font=("Arial", 16))  
44 label_title.pack(pady=10)  
45  
46 # Input Text  
47 entry_text = tk.Entry(root, width=40)  
48 entry_text.pack(pady=5)  
49 entry_text.insert(0, "Enter Text")  
50  
51 # Key Input  
52 entry_key = tk.Entry(root, width=20)  
53 entry_key.pack(pady=5)  
54 entry_key.insert(0, "Enter Key")  
55  
56 # Buttons  
57 btn_encrypt = tk.Button(root, text="Encrypt", command=do_encrypt, bg="green", fg="white")  
58 btn_encrypt.pack(pady=5)  
59
```

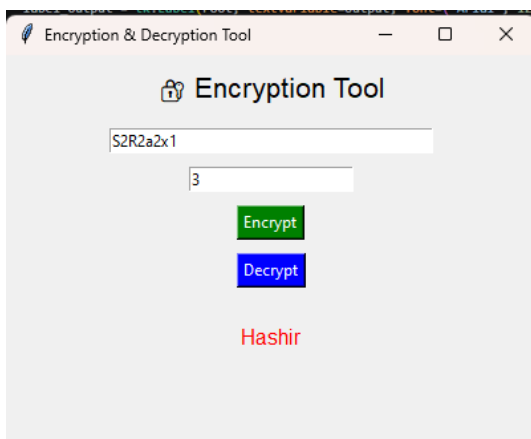
```
45
46 # Input Text
47 entry_text = tk.Entry(root, width=40)
48 entry_text.pack(pady=5)
49 entry_text.insert(0, "Enter Text")
50
51 # Key Input
52 entry_key = tk.Entry(root, width=20)
53 entry_key.pack(pady=5)
54 entry_key.insert(0, "Enter Key")
55
56 # Buttons
57 btn_encrypt = tk.Button(root, text="Encrypt", command=do_encrypt, bg="green", fg="white")
58 btn_encrypt.pack(pady=5)
59
60 btn_decrypt = tk.Button(root, text="Decrypt", command=do_decrypt, bg="blue", fg="white")
61 btn_decrypt.pack(pady=5)
62
63 # Output Label
64 output = tk.StringVar()
65 label_output = tk.Label(root, textvariable=output, font=("Arial", 12), fg="red", wraplength=350)
66 label_output.pack(pady=20)
67
68 # Run App
69 root.mainloop()
```

Output:

Encrypt:



Decrypt:



Result

The encryption tool was successfully tested and worked as expected.

- Plain text was correctly encrypted into unreadable encoded format.
- Decryption successfully restored the original text using the same key.
- Encrypted data was saved and retrieved from a file without data loss.
- Incorrect key usage resulted in failed decryption, ensuring security.
- File handling worked properly for storing and loading encrypted content.

Overall, the system demonstrated proper data transformation, security handling, and file management.

Conclusion:

This project helped in understanding the basics of cryptography, data security, and encryption techniques in Python. By implementing a custom encryption and decryption algorithm, it demonstrates how sensitive data can be protected and safely stored.

It also improved skills in string manipulation, encoding techniques, file handling, and modular programming, making it a simple but effective real-world security simulation tool.

Task 4: System Log Analyzer & Report Generator

Objective

Develop a Python tool that can **analyze system/application log files** and generate meaningful reports. This simulates real-world debugging and monitoring systems used in software companies.

Task Description

Create a CLI-based Python application where users can:

- Load log files (TXT/LOG format)
- Analyze logs for errors, warnings, and info messages
- Count occurrences of different log levels
- Generate a summary report
- Save report to a file

Functional Requirements

Load Log File

- Accept `.txt` or `.log` files
- Read file line by line

Log Analysis

- Detect:
 - ERROR
 - WARNING
 - INFO
- Count frequency of each type
- Extract timestamps (if available)

Search & Filter

- Search logs by keyword
- Filter logs by:
 - Date
 - Log level (ERROR/WARNING/INFO)

Report Generation

- Generate summary:
 - Total logs
 - Total errors
 - Total warnings

- Total info logs
- Save report to TXT/CSV

Menu System

- Load File
- Analyze Logs
- Search Logs
- Generate Report
- Exit

Advanced Requirements (Must Have)

- Use **regular expressions (regex)** for parsing logs
- Implement **error handling** (file not found, empty file)
- Modular code (functions for each feature)
- Case-insensitive search
- Efficient processing for large files

Bonus Features

- Visual report (charts using matplotlib)
- Export report to PDF
- Real-time log monitoring
- Highlight critical errors
- GUI version (Tkinter)

Deliverables

1. ☒ Source Code (ZIP file)
2. ☒ Sample Log File
3. ☒ Generated Report File
4. ☒ GitHub Repository
5. ☒ README.md (features + how to run)
6. ☒ Demo Video (1–2 minutes)



Deadline 24 Hours

Project Objective:

The objective of this project is to develop a Python-based command-line tool that can efficiently analyze system and application log files to support debugging and monitoring tasks. The system is designed to automatically read log files, extract relevant information using regular expressions, and identify log levels such as ERROR, WARNING, and INFO. It aims to provide meaningful insights by counting log occurrences, enabling search and filtering capabilities, and generating summary reports, ultimately simulating real-world log analysis tools used in software development and system maintenance.

Project Description:

The System Log Analyzer & Report Generator is a CLI-based Python application that processes log files in TXT or LOG format and analyzes them to extract structured data such as date, time, log level, and message. The application categorizes logs into ERROR, WARNING, and INFO, calculates their frequency, and allows users to search logs using keywords or filter them based on specific criteria. It also generates a detailed summary report that can be saved to a file for documentation and further analysis. Built using modular programming and proper error handling, the project demonstrates practical implementation of file processing, regex parsing, and data analysis in Python.

Project Created:

ShareView

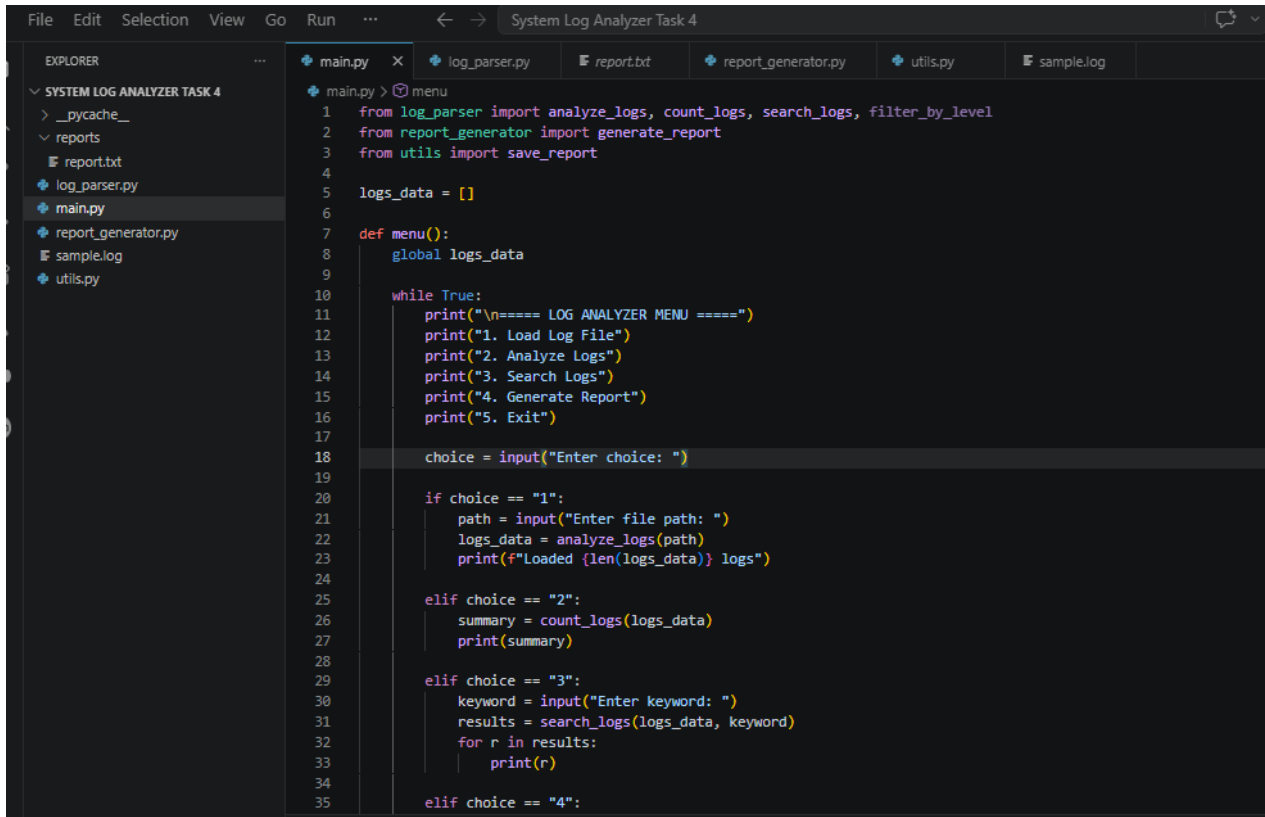
> This PC > New Volume (E:) > Hasnain Karimain (HK) Educational Academy, Software House & Training Center > Hashir Ahmed Task 4,5 > System Log Analyzer Task 4

	Name	Date modified	Type	Size
	 __pycache__	5/6/2026 5:48 PM	File folder	
	 reports	5/6/2026 5:44 PM	File folder	
	log_parser	5/6/2026 5:47 PM	Python Source File	2 KB
	main	5/6/2026 5:42 PM	Python Source File	2 KB
	report_generator	5/6/2026 5:42 PM	Python Source File	1 KB
Certif 	sample	5/6/2026 5:40 PM	Text Document	1 KB
1 has 	utils	5/6/2026 5:42 PM	Python Source File	1 KB

Task

CODE:**Main.py:**

This file is the entry point of the application. It handles the menu system and connects all modules like loading logs, analyzing data, searching, and generating reports.



```
File Edit Selection View Go Run ... System Log Analyzer Task 4
EXPLORER
SYSTEM LOG ANALYZER TASK 4
  > __pycache__
  > reports
  report.txt
  log_parser.py
  main.py
  report_generator.py
  sample.log
  utils.py
main.py > menu
1 from log_parser import analyze_logs, count_logs, search_logs, filter_by_level
2 from report_generator import generate_report
3 from utils import save_report
4
5 logs_data = []
6
7 def menu():
8     global logs_data
9
10    while True:
11        print("\n===== LOG ANALYZER MENU =====")
12        print("1. Load Log File")
13        print("2. Analyze Logs")
14        print("3. Search Logs")
15        print("4. Generate Report")
16        print("5. Exit")
17
18        choice = input("Enter choice: ")
19
20        if choice == "1":
21            path = input("Enter file path: ")
22            logs_data = analyze_logs(path)
23            print(f"Loaded {len(logs_data)} logs")
24
25        elif choice == "2":
26            summary = count_logs(logs_data)
27            print(summary)
28
29        elif choice == "3":
30            keyword = input("Enter keyword: ")
31            results = search_logs(logs_data, keyword)
32            for r in results:
33                print(r)
34
35        elif choice == "4":
```

```
33        print(r)
34
35    elif choice == "4":
36        summary = count_logs(logs_data)
37        report = generate_report(summary, logs_data)
38        save_report(report)
39
40    elif choice == "5":
41        print("Exiting...")
42        break
43
44    else:
45        print("Invalid choice!")
46
47    menu()
```

Log_Parser.py:

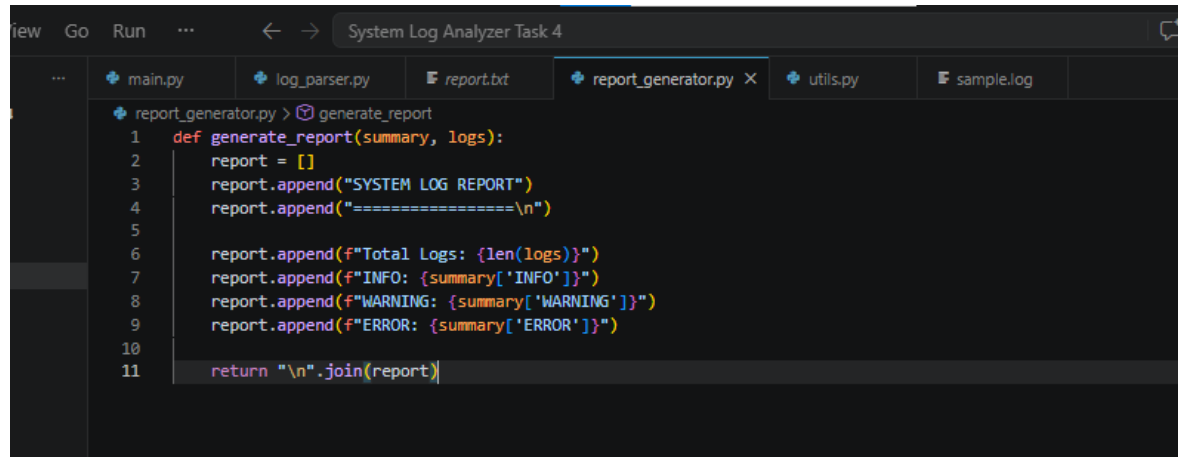
This file is responsible for reading log files and parsing each line using regular expressions to extract structured data like date, time, log level, and message.

```
1 import re
2
3 LOG_PATTERN = r'(?P<date>\d{4}-\d{2}-\d{2}) (?P<time>\d{2}:\d{2}:\d{2}) (?P<level>INFO|ERROR|WARNING) (?P<message>.*)'
4
5 def parse_log_line(line):
6     match = re.match(LOG_PATTERN, line)
7     if match:
8         return match.groupdict()
9     return None
10
11 def analyze_logs(file_path):
12     logs = []
13
14     try:
15         with open(file_path, 'r') as file:
16             for line in file:
17                 parsed = parse_log_line(line.strip())
18                 if parsed:
19                     logs.append(parsed)
20     except FileNotFoundError:
21         print("X File not found!")
22         return []
23
24     return logs
25
26 def count_logs(logs):
27     summary = {
28         "INFO": 0,
29         "ERROR": 0,
30         "WARNING": 0
31     }
32
33     for log in logs:
34         level = log["level"]
35         summary[level] += 1
```

```
31 }
32
33 for log in logs:
34     level = log["level"]
35     summary[level] += 1
36
37 return summary
38
39 def search_logs(logs, keyword):
40     result = []
41
42     keyword = keyword.lower()
43
44     for log in logs:
45         if (keyword in log["message"].lower() or
46             keyword in log["level"].lower() or
47             keyword in log["date"] or
48             keyword in log["time"]):
49             result.append(log)
50
51     return result
52
53 def filter_by_level(logs, level):
54     return [log for log in logs if log["level"] == level]
```

Report_generator.py:

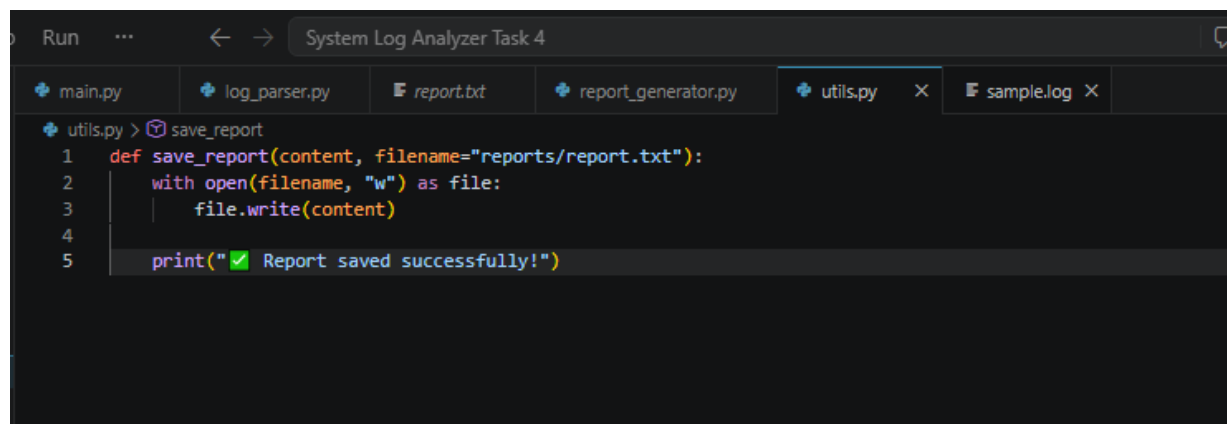
This file generates a clean summary report based on analyzed log data, showing total logs and counts of ERROR, WARNING, and INFO.



```
view Go Run ... < -> System Log Analyzer Task 4
...
main.py log_parser.py report.txt report_generator.py x utils.py sample.log
report_generator.py > generate_report
1 def generate_report(summary, logs):
2     report = []
3     report.append("SYSTEM LOG REPORT")
4     report.append("=====\n")
5
6     report.append(f"Total Logs: {len(logs)}")
7     report.append(f"INFO: {summary['INFO']}")
8     report.append(f"WARNING: {summary['WARNING']}")
9     report.append(f"ERROR: {summary['ERROR']}")
10
11     return "\n".join(report)
```

Utils.py:

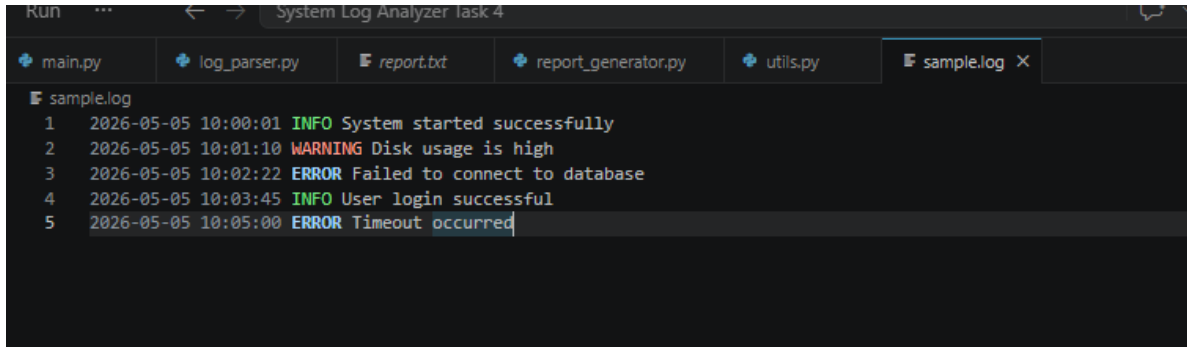
This file contains helper functions such as saving the generated report into a text file for future use and documentation.



```
Run ... < -> System Log Analyzer Task 4
main.py log_parser.py report.txt report_generator.py utils.py x sample.log x
utils.py > save_report
1 def save_report(content, filename="reports/report.txt"):
2     with open(filename, "w") as file:
3         file.write(content)
4
5     print("✅ Report saved successfully!")
```

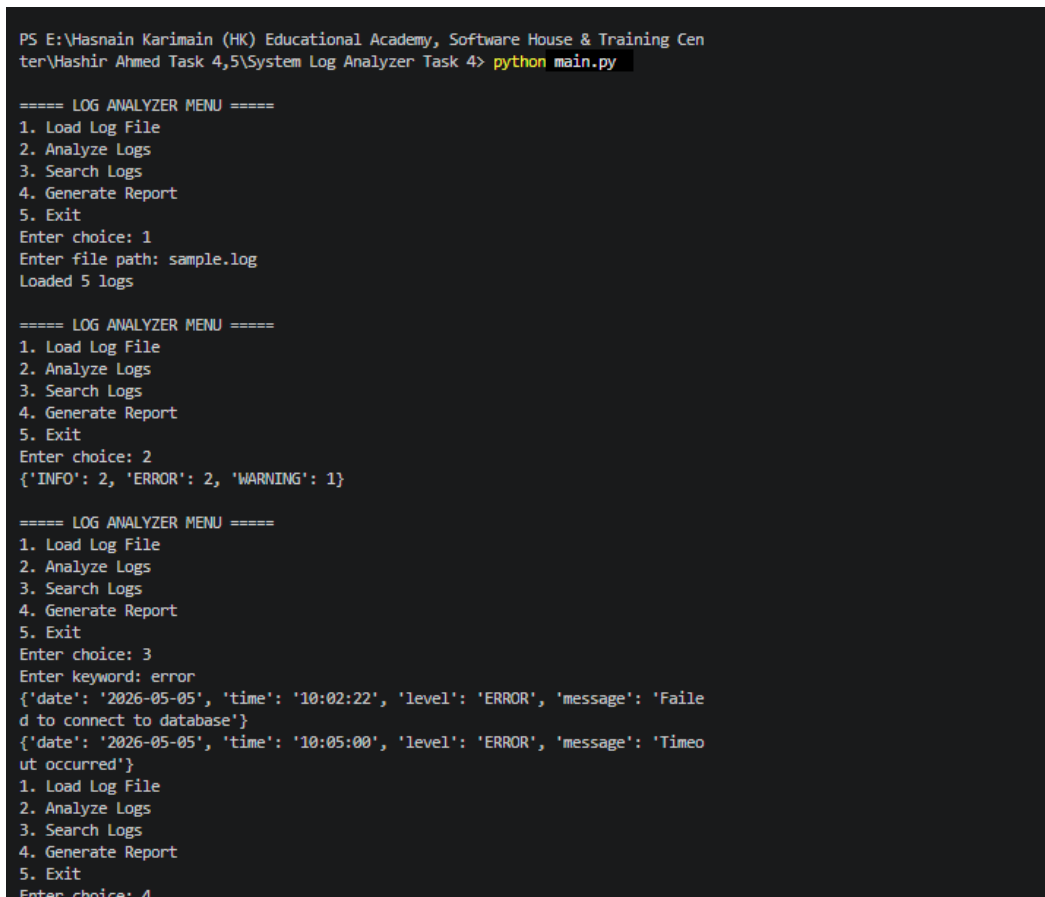
Sample.log:

This is a sample log file used for testing the application. It contains different types of log entries like INFO, WARNING, and ERROR for analysis.



```
1 2026-05-05 10:00:01 INFO System started successfully
2 2026-05-05 10:01:10 WARNING Disk usage is high
3 2026-05-05 10:02:22 ERROR Failed to connect to database
4 2026-05-05 10:03:45 INFO User login successful
5 2026-05-05 10:05:00 ERROR Timeout occurred
```

OUTPUT:



```
PS E:\Hasnain Karimain (HK) Educational Academy, Software House & Training Cen
ter\Hashir Ahmed Task 4,5\System Log Analyzer Task 4> python main.py

===== LOG ANALYZER MENU =====
1. Load Log File
2. Analyze Logs
3. Search Logs
4. Generate Report
5. Exit
Enter choice: 1
Enter file path: sample.log
Loaded 5 logs

===== LOG ANALYZER MENU =====
1. Load Log File
2. Analyze Logs
3. Search Logs
4. Generate Report
5. Exit
Enter choice: 2
{'INFO': 2, 'ERROR': 2, 'WARNING': 1}

===== LOG ANALYZER MENU =====
1. Load Log File
2. Analyze Logs
3. Search Logs
4. Generate Report
5. Exit
Enter choice: 3
Enter keyword: error
{'date': '2026-05-05', 'time': '10:02:22', 'level': 'ERROR', 'message': 'Faile
d to connect to database'}
{'date': '2026-05-05', 'time': '10:05:00', 'level': 'ERROR', 'message': 'Timeo
ut occurred'}
1. Load Log File
2. Analyze Logs
3. Search Logs
4. Generate Report
5. Exit
Enter choice: 4
```

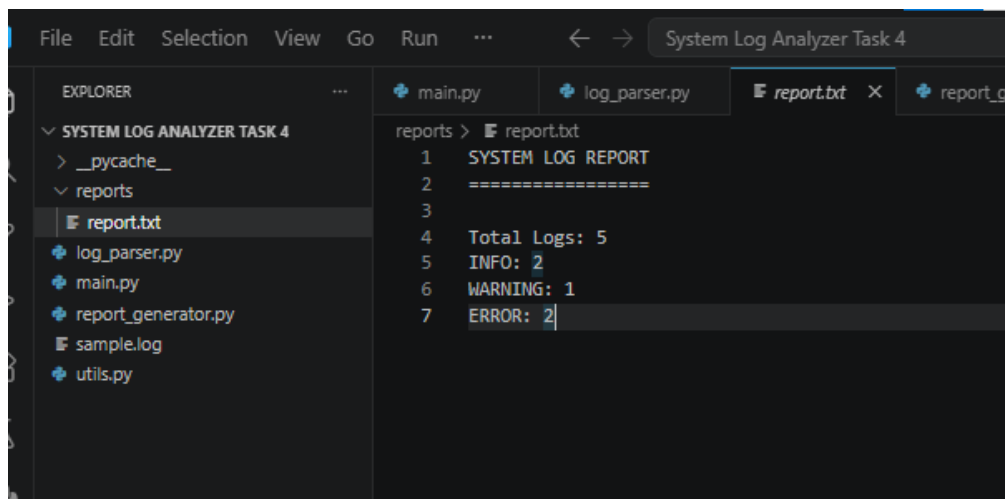
```
===== LOG ANALYZER MENU =====
1. Load Log File
2. Analyze Logs
3. Search Logs
4. Generate Report
5. Exit
Enter choice: 3
Enter keyword: error
{'date': '2026-05-05', 'time': '10:02:22', 'level': 'ERROR', 'message': 'Failed to connect to database'}
{'date': '2026-05-05', 'time': '10:05:00', 'level': 'ERROR', 'message': 'Timeout occurred'}
1. Load Log File
2. Analyze Logs
3. Search Logs
4. Generate Report
5. Exit
Enter choice: 4
✅ Report saved successfully!

===== LOG ANALYZER MENU =====
1. Load Log File
2. Analyze Logs
3. Search Logs
4. Generate Report
5. Exit
Enter choice: 5
Exiting...
PS E:\Hasnain Karimain (HK) Educational Academy, Software House & Training Cen
```

Report Result

The generated report shows the total number of logs along with a breakdown of log levels including ERROR, WARNING, and INFO. It helps in quickly identifying system behavior and detecting issues by summarizing all log activities in a structured format.

Report.txt:



The screenshot shows a code editor with the following files in the Explorer: main.py, log_parser.py, report.txt, report_generator.py, sample.log, and utils.py. The report.txt file is open, displaying the following content:

```
reports > report.txt
1  SYSTEM LOG REPORT
2  =====
3
4  Total Logs: 5
5  INFO: 2
6  WARNING: 1
7  ERROR: 2
```

Conclusion

The System Log Analyzer & Report Generator is a powerful and practical Python-based CLI tool that automates the process of reading, analyzing, and summarizing system log files. It efficiently extracts important information such as timestamps, log levels, and messages using regular expressions, and categorizes logs into ERROR, WARNING, and INFO for better understanding of system behavior. The project also provides features like searching, filtering, and generating structured reports, which help in quickly identifying issues and monitoring system performance. Overall, this project demonstrates real-world application of file handling, data processing, and modular programming in Python, making it highly useful for debugging and system analysis tasks.

Task 5: Automated Bulk Email Sender with Templates

Objective

Develop a Python-based system to **send bulk emails automatically** using templates. This simulates real-world email marketing and notification systems.

Task Description

Create a CLI-based (or optional GUI) Python application where users can:

- Send emails to multiple recipients
- Use predefined email templates
- Personalize emails (Name, Company, etc.)
- Load recipient list from CSV/JSON
- Track sent email history

Functional Requirements

Email Sending

- Use SMTP (`smtplib`)
- Input:
 - Sender email & password/app password
 - Recipient email(s)
 - Subject
 - Message

Bulk Sending

- Load contacts from CSV/JSON
- Send emails to multiple users automatically

Templates

- Create reusable email templates
- Use placeholders like:
 - `{name}`
 - `{company}`
- Replace dynamically for each user

Email Personalization

- Customize each email based on user data

- Example:
 - "Hello Ali, welcome to HK Academy"

◆ Email History

- Save sent emails in JSON/CSV
- Store:
 - Email
 - Subject
 - Status (Sent/Failed)
 - Timestamp

◆ Menu System

- Send Email
- Load Contacts
- Choose Template
- View History
- Exit

💡 Advanced Requirements (Must Have)

- Proper **error handling** (invalid email, connection error)
- Modular code structure
- Secure login (use app password)
- Retry mechanism for failed emails
- Case-insensitive email handling

✿ Bonus Features

- Attach files (PDF, images)
- HTML email templates (styled emails)
- Schedule emails (send later)
- GUI version (Tkinter)
- Email open tracking (advanced)

🕒 Deadline 24 Hours

📦 Deliverables

1. ☒ Source Code (ZIP file)
2. ☒ Sample CSV/JSON contact file
3. ☒ Email Template File
4. ☒ Email History File
5. ☒ GitHub Repository

6. ☒ README.md (setup + SMTP guide)
7. ☒ Demo Video (1–2 minutes)

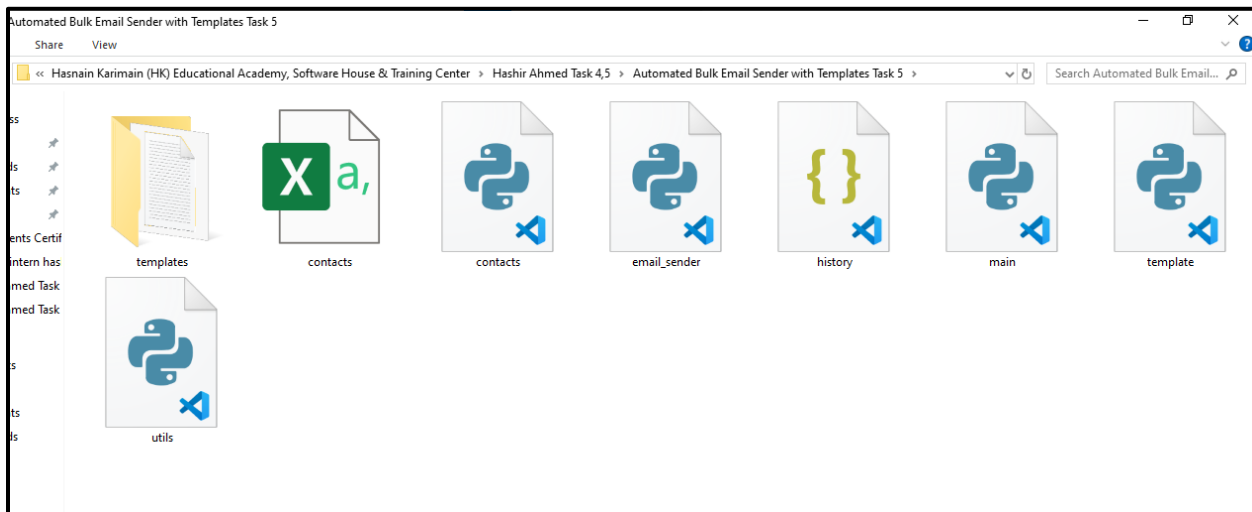
Project Objective

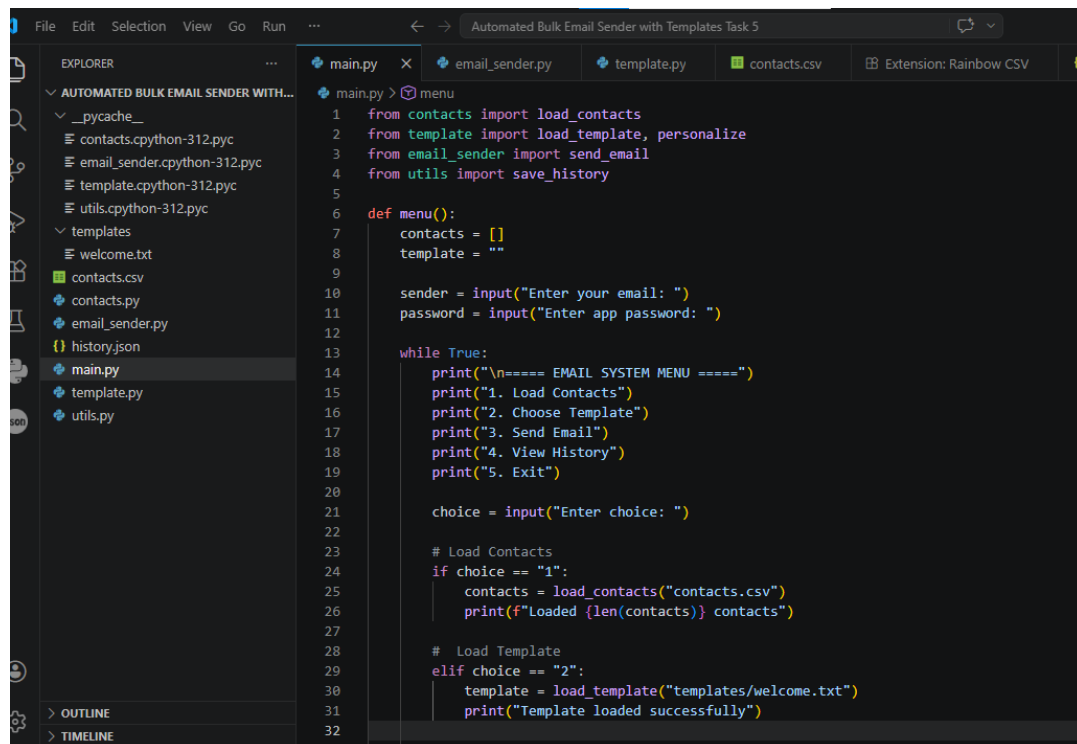
The objective of this project is to develop a Python-based automated bulk email sending system that can send personalized emails to multiple recipients using SMTP. It aims to simulate real-world email marketing and notification systems by allowing users to load contacts, use templates with dynamic placeholders, and track email delivery status efficiently.

Project Description

The Automated Bulk Email Sender with Templates is a CLI-based Python application that enables users to send emails to multiple recipients automatically. It reads contact data from a CSV file, uses reusable email templates with placeholders like {name} and {company}, and personalizes each email before sending. The system uses SMTP for email delivery, includes error handling for failed emails, and stores email history (sent/failed status with timestamps) in a JSON file for tracking and record-keeping.

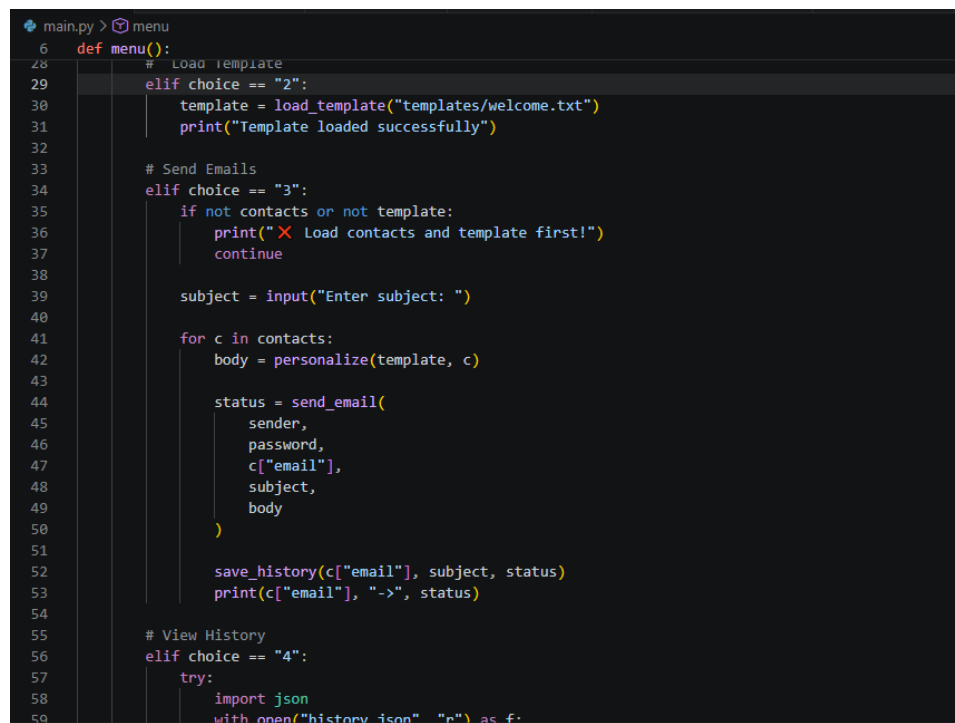
Project File Created:



CODE:**Main.py:**

```
File Edit Selection View Go Run ... Automated Bulk Email Sender with Templates Task 5
EXPLORER
AUTOMATED BULK EMAIL SENDER WITH...
  __pycache__
    contacts.cpython-312.pyc
    email_sender.cpython-312.pyc
    template.cpython-312.pyc
    utils.cpython-312.pyc
  templates
    welcome.txt
  contacts.csv
  contacts.py
  email_sender.py
  history.json
  main.py
  template.py
  utils.py
OUTLINE
TIMELINE

main.py > menu
1 from contacts import load_contacts
2 from template import load_template, personalize
3 from email_sender import send_email
4 from utils import save_history
5
6 def menu():
7     contacts = []
8     template = ""
9
10    sender = input("Enter your email: ")
11    password = input("Enter app password: ")
12
13    while True:
14        print("\n==== EMAIL SYSTEM MENU =====")
15        print("1. Load Contacts")
16        print("2. Choose Template")
17        print("3. Send Email")
18        print("4. View History")
19        print("5. Exit")
20
21        choice = input("Enter choice: ")
22
23        # Load Contacts
24        if choice == "1":
25            contacts = load_contacts("contacts.csv")
26            print(f"Loaded {len(contacts)} contacts")
27
28        # Load Template
29        elif choice == "2":
30            template = load_template("templates/welcome.txt")
31            print("Template loaded successfully")
32
```



```
main.py > menu
6 def menu():
28     # Load template
29     elif choice == "2":
30         template = load_template("templates/welcome.txt")
31         print("Template loaded successfully")
32
33     # Send Emails
34     elif choice == "3":
35         if not contacts or not template:
36             print("X Load contacts and template first!")
37             continue
38
39         subject = input("Enter subject: ")
40
41         for c in contacts:
42             body = personalize(template, c)
43
44             status = send_email(
45                 sender,
46                 password,
47                 c["email"],
48                 subject,
49                 body
50             )
51
52             save_history(c["email"], subject, status)
53             print(c["email"], "->", status)
54
55     # View History
56     elif choice == "4":
57         try:
58             import json
59             with open("history.json" "r") as f:
```

```
54
55 # View History
56 elif choice == "4":
57     try:
58         import json
59         with open("history.json", "r") as f:
60             data = json.load(f)
61             for h in data:
62                 print(h)
63     except:
64         print("No history found")
65
66 # Exit
67 elif choice == "5":
68     print("Exiting...")
69     break
70
71 else:
72     print("Invalid choice!")
73
74 menu()
```

Email_sender.py:

```
Go Run ... Automated Bulk Email Sender with Templates Task 5
main.py email_sender.py x template.py contacts.csv Extension: Rainbow CSV
email_sender.py > send_email
1 import smtplib
2 from email.mime.text import MIMEText
3
4 def send_email(sender, password, receiver, subject, body):
5     try:
6         msg = MIMEText(body)
7         msg['Subject'] = subject
8         msg['From'] = sender
9         msg['To'] = receiver
10
11         server = smtplib.SMTP('smtp.gmail.com', 587)
12         server.starttls()
13         server.login(sender, password)
14
15         server.send_message(msg)
16         server.quit()
17
18         return "Sent"
19     except Exception as e:
20         return f"Failed: {str(e)}"
```

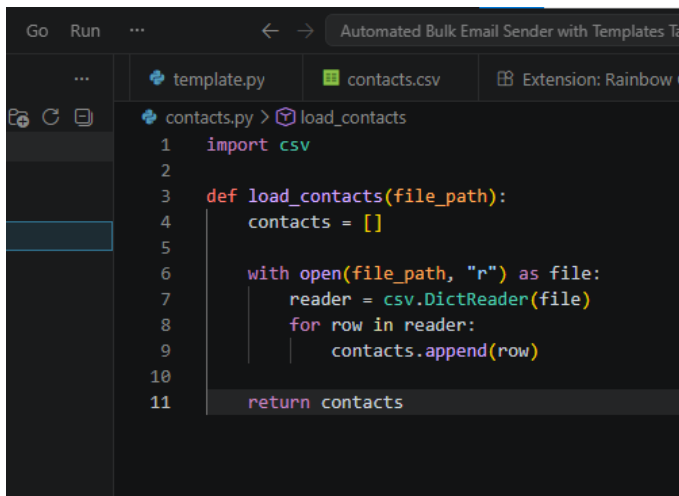
Template.py:

```
... Automated Bulk Email Sender with Templates Task 5
main.py email_sender.py template.py X contacts.csv
template.py > personalize
1 def load_template(path):
2     with open(path, "r") as file:
3         return file.read()
4
5
6 def personalize(template, contact):
7     return template.format(
8         name=contact["name"],
9         company=contact["company"]
10    )
```

Utils.py:

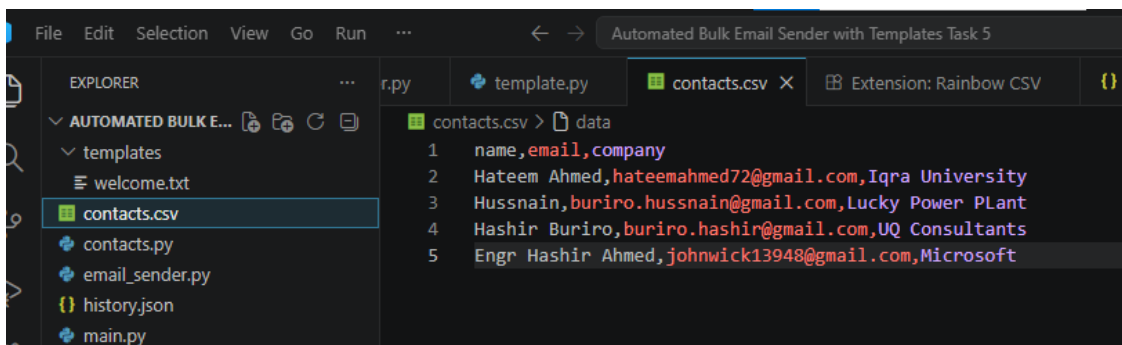
```
TH... Automated Bulk Email Sender with Templates Task 5
main.py email_sender.py template.py contacts.csv Extension: Rainbow
utils.py > save_history
1 import json
2 from datetime import datetime
3
4 def save_history(email, subject, status):
5     record = {
6         "email": email,
7         "subject": subject,
8         "status": status,
9         "time": str(datetime.now())
10    }
11
12    try:
13        with open("history.json", "r") as file:
14            data = json.load(file)
15    except:
16        data = []
17
18    data.append(record)
19
20    with open("history.json", "w") as file:
21        json.dump(data, file, indent=4)
```

Contacts.py:



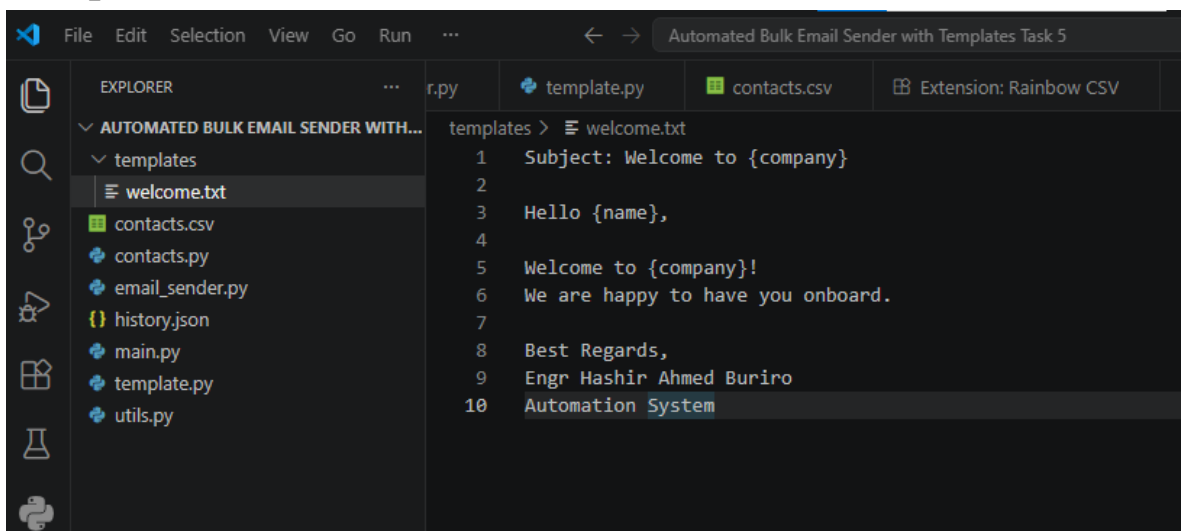
```
Go Run ... Automated Bulk Email Sender with Templates Task 5
template.py contacts.csv Extension: Rainbow CSV
contacts.py > load_contacts
1 import csv
2
3 def load_contacts(file_path):
4     contacts = []
5
6     with open(file_path, "r") as file:
7         reader = csv.DictReader(file)
8         for row in reader:
9             contacts.append(row)
10
11     return contacts
```

Contacts.csv:

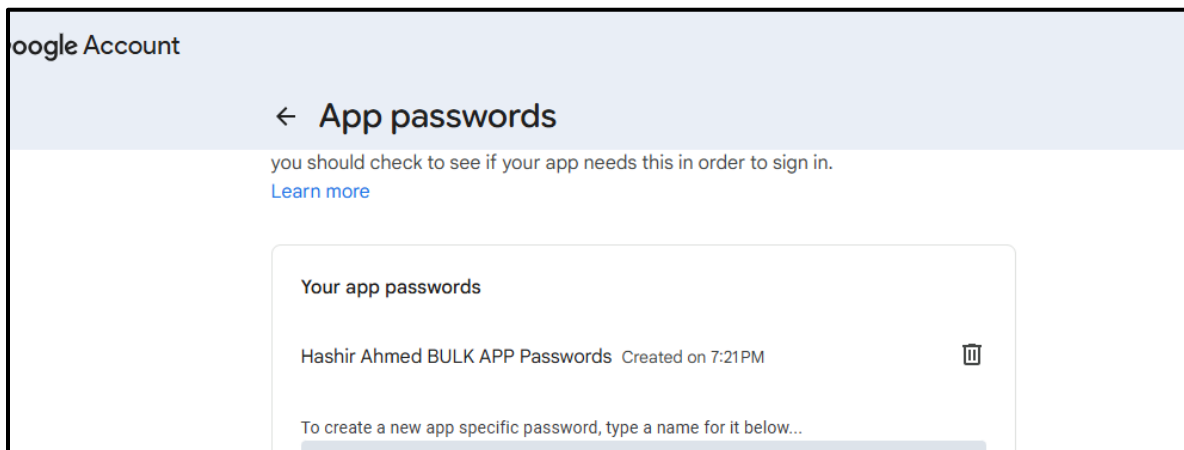
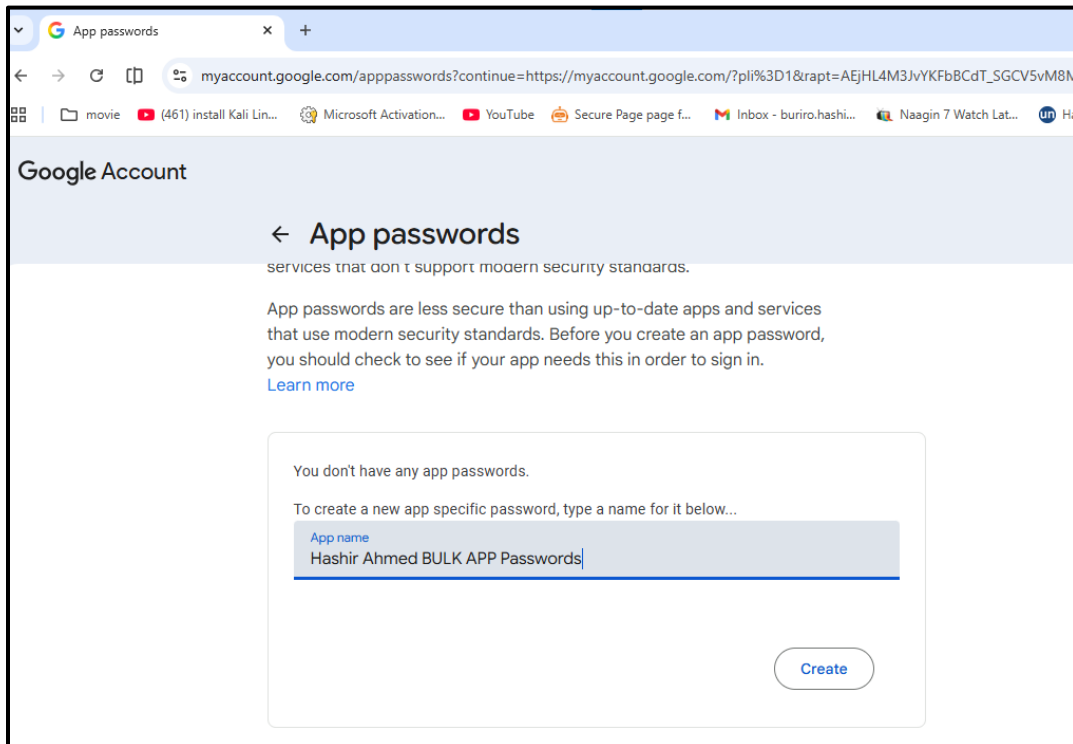


```
File Edit Selection View Go Run ... Automated Bulk Email Sender with Templates Task 5
EXPLORER
AUTOMATED BULK E...
templates
welcome.txt
contacts.csv
contacts.py
email_sender.py
history.json
main.py
template.py
r.py contacts.csv X Extension: Rainbow CSV {} h
contacts.csv > data
1 name,email,company
2 Hateem Ahmed,hateemahmed72@gmail.com,Iqra University
3 Hussnain,buriro.hussnain@gmail.com,Lucky Power Plant
4 Hashir Buriro,buriro.hashir@gmail.com,UQ Consultants
5 Engr Hashir Ahmed,johnwick13948@gmail.com,Microsoft
```

Templates/welcome.txt:



```
File Edit Selection View Go Run ... Automated Bulk Email Sender with Templates Task 5
EXPLORER
AUTOMATED BULK EMAIL SENDER WITH...
templates
welcome.txt
contacts.csv
contacts.py
email_sender.py
history.json
main.py
template.py
utils.py
r.py template.py contacts.csv Extension: Rainbow CSV {}
templates > welcome.txt
1 Subject: Welcome to {company}
2
3 Hello {name},
4
5 Welcome to {company}!
6 We are happy to have you onboard.
7
8 Best Regards,
9 Engr Hashir Ahmed Buriro
10 Automation System
```

BULK APP CREATED in Google App Passwords:

OUTPUT:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS E:\Hasnain Karimain (HK) Educational Academy, S
ask 5> python main.py
Enter your email: buriro.hashir@gmail.com
Enter app password: gkes pwtm iggz cslk

===== EMAIL SYSTEM MENU =====
1. Load Contacts
2. Choose Template
3. Send Email
4. View History
5. Exit
Enter choice: 1
Loaded 4 contacts

===== EMAIL SYSTEM MENU =====
1. Load Contacts
2. Choose Template
3. Send Email
4. View History
5. Exit
Enter choice: 2
Template loaded successfully

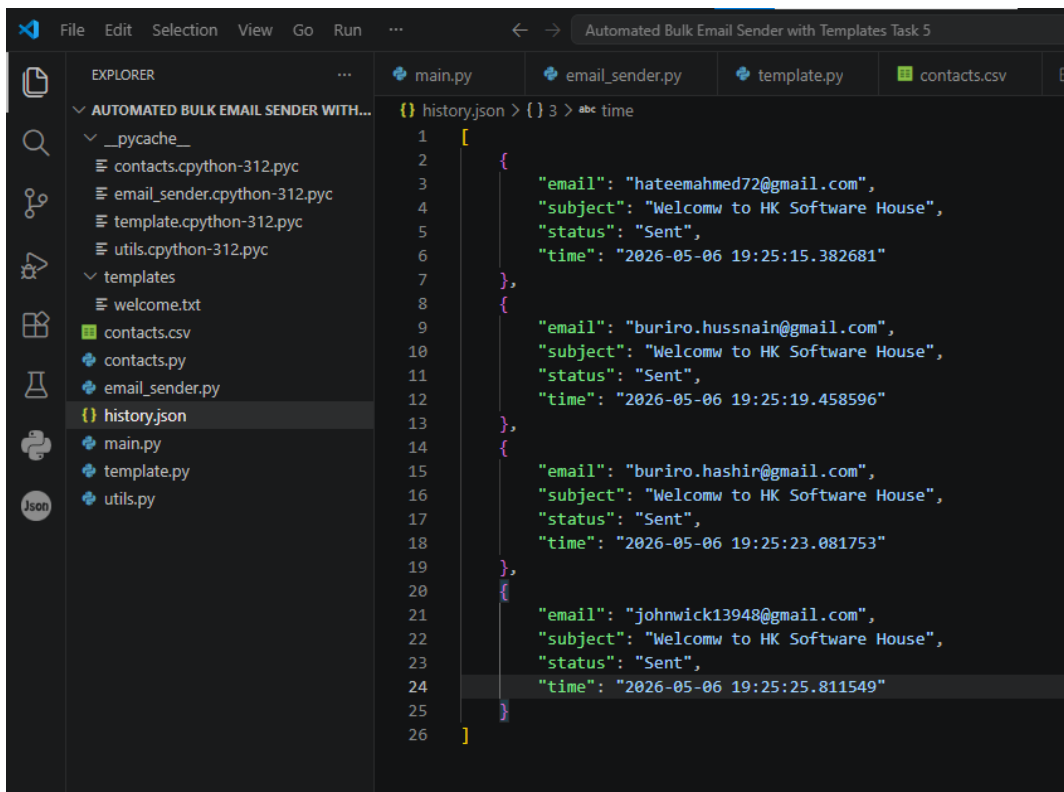
===== EMAIL SYSTEM MENU =====
1. Load Contacts
2. Choose Template
3. Send Email
4. View History
5. Exit
Enter choice: 3
Enter subject: Welcomw to HK Software House
hateemahmed72@gmail.com -> Sent
buriro.hussnain@gmail.com -> Sent
buriro.hashir@gmail.com -> Sent
johnwick13948@gmail.com -> Sent

===== EMAIL SYSTEM MENU =====
```

```
===== EMAIL SYSTEM MENU =====
1. Load Contacts
2. Choose Template
3. Send Email
4. View History
5. Exit
Enter choice: 4
{'email': 'hateemahmed72@gmail.com', 'subject': 'W
elcomw to HK Software House', 'status': 'Sent', 't
ime': '2026-05-06 19:25:15.382681'}
{'email': 'buriro.hussnain@gmail.com', 'subject':
'Welcomw to HK Software House', 'status': 'Sent',
'time': '2026-05-06 19:25:19.458596'}
{'email': 'buriro.hashir@gmail.com', 'subject': 'W
elcomw to HK Software House', 'status': 'Sent', 't
ime': '2026-05-06 19:25:23.081753'}
{'email': 'johnwick13948@gmail.com', 'subject': 'W
elcomw to HK Software House', 'status': 'Sent', 't
ime': '2026-05-06 19:25:25.811549'}

===== EMAIL SYSTEM MENU =====
1. Load Contacts
2. Choose Template
3. Send Email
4. View History
5. Exit
Enter choice: 5
Exiting...
PS E:\Hasnain Karimain (HK) Educational Academy, S
oftware House & Training Center\Hashir Ahmed Task
4,5\Automated Bulk Email Sender with Templates Tas
k 5> |
```

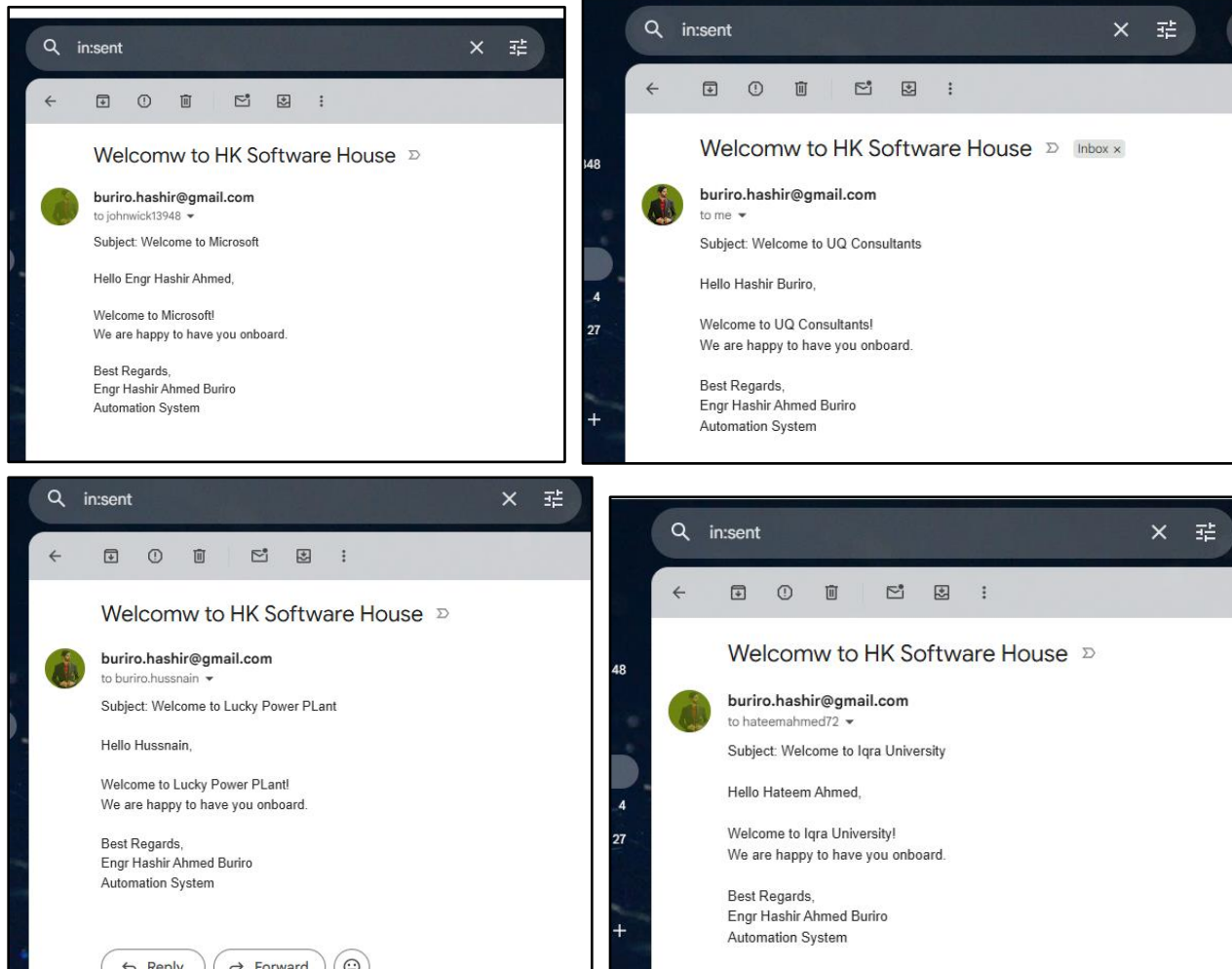
History.json:



The screenshot shows a Visual Studio Code editor window with the file explorer on the left and the code editor on the right. The file explorer shows a project named 'AUTOMATED BULK EMAIL SENDER WITH...' with various files and folders. The code editor displays the content of 'history.json', which is a JSON array of email records. The records include email addresses, subjects, statuses, and timestamps.

```
{
  "email": "hateemahmed72@gmail.com",
  "subject": "Welcomw to HK Software House",
  "status": "Sent",
  "time": "2026-05-06 19:25:15.382681"
},
{
  "email": "buriro.hussnain@gmail.com",
  "subject": "Welcomw to HK Software House",
  "status": "Sent",
  "time": "2026-05-06 19:25:19.458596"
},
{
  "email": "buriro.hashir@gmail.com",
  "subject": "Welcomw to HK Software House",
  "status": "Sent",
  "time": "2026-05-06 19:25:23.081753"
},
{
  "email": "johnwick13948@gmail.com",
  "subject": "Welcomw to HK Software House",
  "status": "Sent",
  "time": "2026-05-06 19:25:25.811549"
}
```

All mails were sent successfully:



Result

The project successfully sends emails to all recipients listed in the CSV file and replaces template placeholders with user-specific data. After execution, it displays the delivery status of each email (Sent/Failed) in the terminal and stores a complete history of all email activities in a JSON file, making it easy to verify successful delivery and monitor failures.

Conclusion

This project demonstrates a practical implementation of automation using Python for bulk email sending with personalization features. It highlights key programming concepts such as file handling, SMTP email communication, template processing, error handling, and data storage. Overall, it provides a strong foundation for understanding real-world email automation systems used in marketing, notifications, and business communication.

Task 6: PDF Report Generator for Student/Company Data

Objective

Develop a Python tool to generate professional PDF reports for student or company data. This simulates real-world reporting systems used in businesses and institutions.

Task Description

Create a CLI-based (or optional GUI) Python application where users can:

- Input student/company data
- Generate structured PDF reports
- Format data professionally (tables, headings)
- Save reports automatically

Functional Requirements

Data Input

- Accept input:
 - Name
 - ID
 - Email
 - Course/Department or Company Role
 - Performance/Details
- Option to load data from CSV/JSON

PDF Generation

- Generate PDF using [reportlab](#)
- Include:
 - Title (Report Name)
 - User details
 - Table format data
 - Date & time

◆ Report Types

- Student Report (marks, attendance, etc.)
- Company Report (employee summary, performance)

◆ File Saving

- Save PDF with unique filename
- Store in a specific folder

◆ Menu System

- Add Data
- Load Data
- Generate PDF
- Exit

💡 Advanced Requirements (Must Have)

- Proper formatting (headings, spacing)
- Use tables in PDF
- Modular code structure
- Error handling (missing data, file issues)
- Dynamic data support (multiple entries)

✳ Bonus Features

- Add logo to PDF (company branding)
- Export multiple reports at once
- Add charts/graphs
- Password-protected PDF
- GUI version (Tkinter)

📦 Deliverables

1. Source Code (ZIP file)
2. Sample Data File (CSV/JSON)
3. Generated PDF Report
4. GitHub Repository
5. README.md (how to run + features)
6. Demo Video (1–2 minutes)

🕒 Deadline 24 Hours

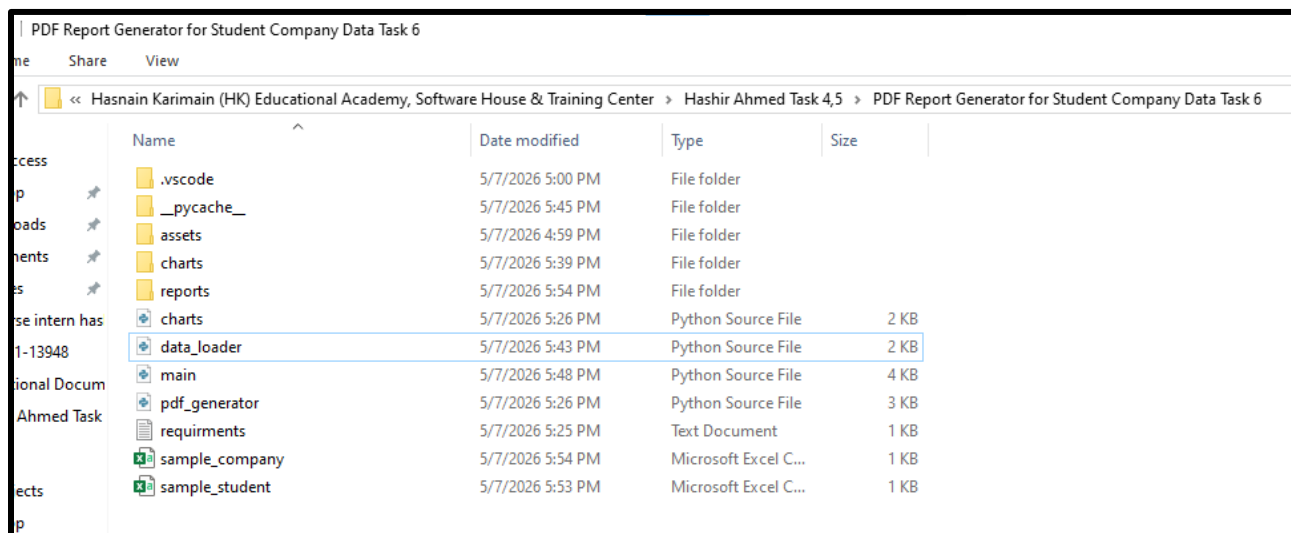
Project Objective

The objective of this project is to develop a professional Python-based PDF Report Generator that allows users to input student or company data and automatically generate well-formatted PDF reports. The system supports dynamic multiple entries, table formatting, CSV/JSON data loading, and automatic file saving. This project simulates real-world reporting systems used in schools, universities, and companies.

Project Description

This project is a CLI-based Python application developed using the `reportlab` library. It allows users to add student or employee data manually or load data from CSV/JSON files. The system generates structured PDF reports containing headings, tables, date & time, and user details. The application follows a modular coding structure with proper error handling and automatic report saving. It can be extended further with GUI, charts, logos, and password protection.

Project Created:

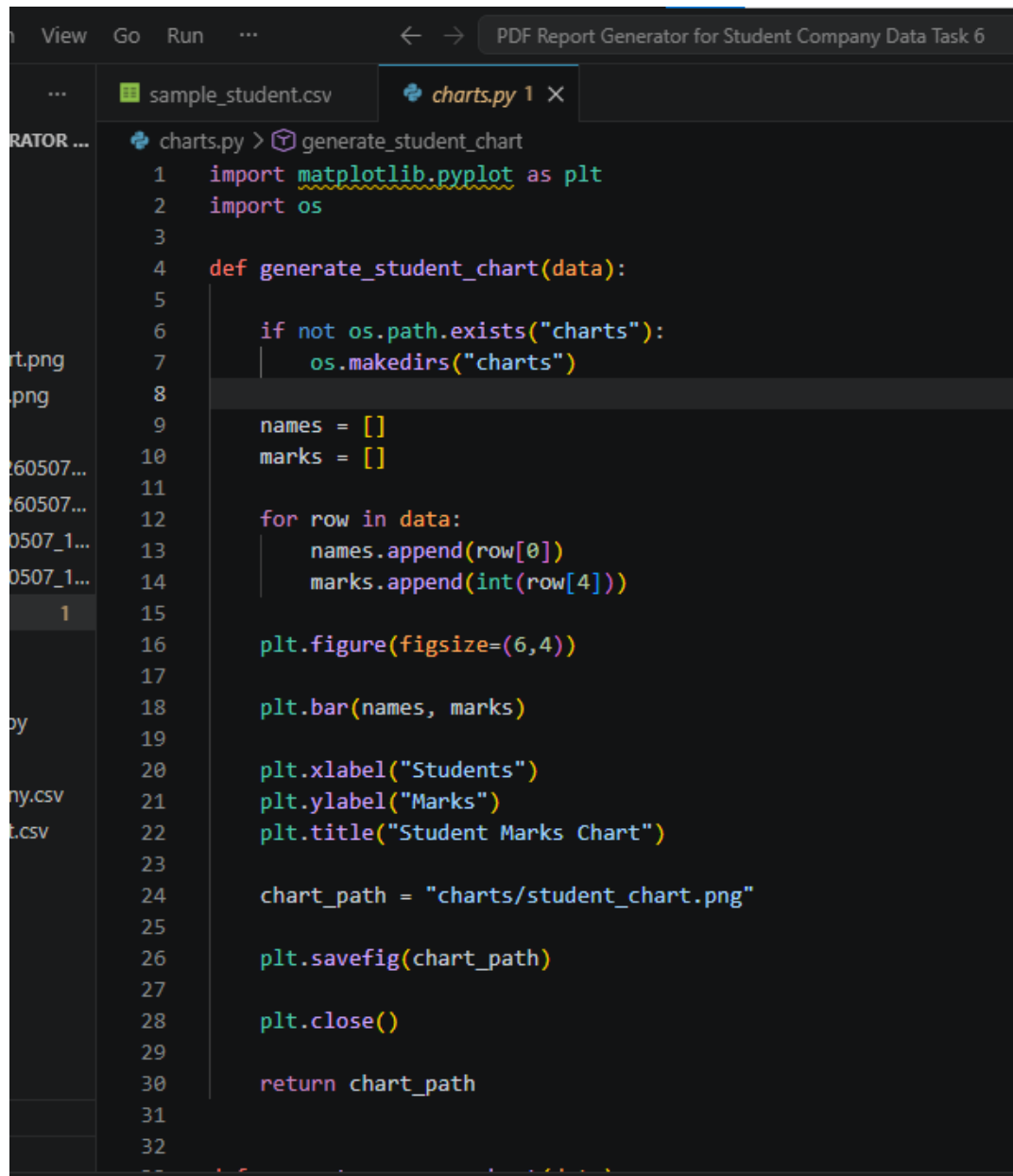


The screenshot shows a file explorer window titled "PDF Report Generator for Student Company Data Task 6". The breadcrumb path is: << Hasnain Karimain (HK) Educational Academy, Software House & Training Center > Hashir Ahmed Task 4,5 > PDF Report Generator for Student Company Data Task 6. The table below lists the files and folders in the project directory.

Name	Date modified	Type	Size
.vscode	5/7/2026 5:00 PM	File folder	
__pycache__	5/7/2026 5:45 PM	File folder	
assets	5/7/2026 4:59 PM	File folder	
charts	5/7/2026 5:39 PM	File folder	
reports	5/7/2026 5:54 PM	File folder	
charts	5/7/2026 5:26 PM	Python Source File	2 KB
data_loader	5/7/2026 5:43 PM	Python Source File	2 KB
main	5/7/2026 5:48 PM	Python Source File	4 KB
pdf_generator	5/7/2026 5:26 PM	Python Source File	3 KB
requirements	5/7/2026 5:25 PM	Text Document	1 KB
sample_company	5/7/2026 5:54 PM	Microsoft Excel C...	1 KB
sample_student	5/7/2026 5:53 PM	Microsoft Excel C...	1 KB

CODE;

Charts.py:

The image is a screenshot of a code editor window. The title bar at the top reads 'PDF Report Generator for Student Company Data Task 6'. Below the title bar, there are two tabs: 'sample_student.csv' and 'charts.py 1 X'. The 'charts.py' tab is active, showing the following Python code:

```
1 import matplotlib.pyplot as plt
2 import os
3
4 def generate_student_chart(data):
5
6     if not os.path.exists("charts"):
7         os.makedirs("charts")
8
9     names = []
10    marks = []
11
12    for row in data:
13        names.append(row[0])
14        marks.append(int(row[4]))
15
16    plt.figure(figsize=(6,4))
17
18    plt.bar(names, marks)
19
20    plt.xlabel("Students")
21    plt.ylabel("Marks")
22    plt.title("Student Marks Chart")
23
24    chart_path = "charts/student_chart.png"
25
26    plt.savefig(chart_path)
27
28    plt.close()
29
30    return chart_path
31
32
```

```
charts.py > ...
32
33 def generate_company_chart(data):
34
35     if not os.path.exists("charts"):
36         os.makedirs("charts")
37
38     performance_count = {
39         "Excellent": 0,
40         "Good": 0,
41         "Average": 0
42     }
43
44     for row in data:
45
46         performance = row[5]
47
48         if performance in performance_count:
49             performance_count[performance] += 1
50
51     plt.figure(figsize=(6,4))
52
53     plt.bar(
54         performance_count.keys(),
55         performance_count.values()
56     )
57
58     plt.xlabel("Performance")
59     plt.ylabel("Employees")
60     plt.title("Company Performance Chart")
61
62     chart_path = "charts/company_chart.png"
63
```

```
60     plt.title("Company Performance Chart")
61
62     chart_path = "charts/company_chart.png"
63
64     plt.savefig(chart_path)
65
66     plt.close()
67
68     return chart_path
```

Data_loader.py:

```
26 def save_student_data(data):
29     sample_student.csv
30 )
31
32 with open(
33     "sample_student.csv",
34     mode="a",
35     newline=""
36 ) as file:
37
38     writer = csv.writer(file)
39
40     # Header
41     if not file_exists:
42
43         writer.writerow([
44             "Name",
45             "ID",
46             "Email",
47             "Course",
48             "Marks",
49             "Attendance"
50         ])
51
52     writer.writerow(data)
53
54 # =====
55 # SAVE COMPANY DATA
56 # =====
57
58 def save_company_data(data):
59
60     file_exists = os.path.isfile(
```

```
data_loader.py > save_company_data
57
58 def save_company_data(data):
59
60     file_exists = os.path.isfile(
61         "sample_company.csv"
62     )
63
64     with open(
65         "sample_company.csv",
66         mode="a",
67         newline=""
68     ) as file:
69
70         writer = csv.writer(file)
71
72         # Header
73         if not file_exists:
74
75             writer.writerow([
76                 "Name",
77                 "ID",
78                 "Email",
79                 "Department",
80                 "Role",
81                 "Performance"
82             ])
83
84         writer.writerow(data)
```

Main.py:

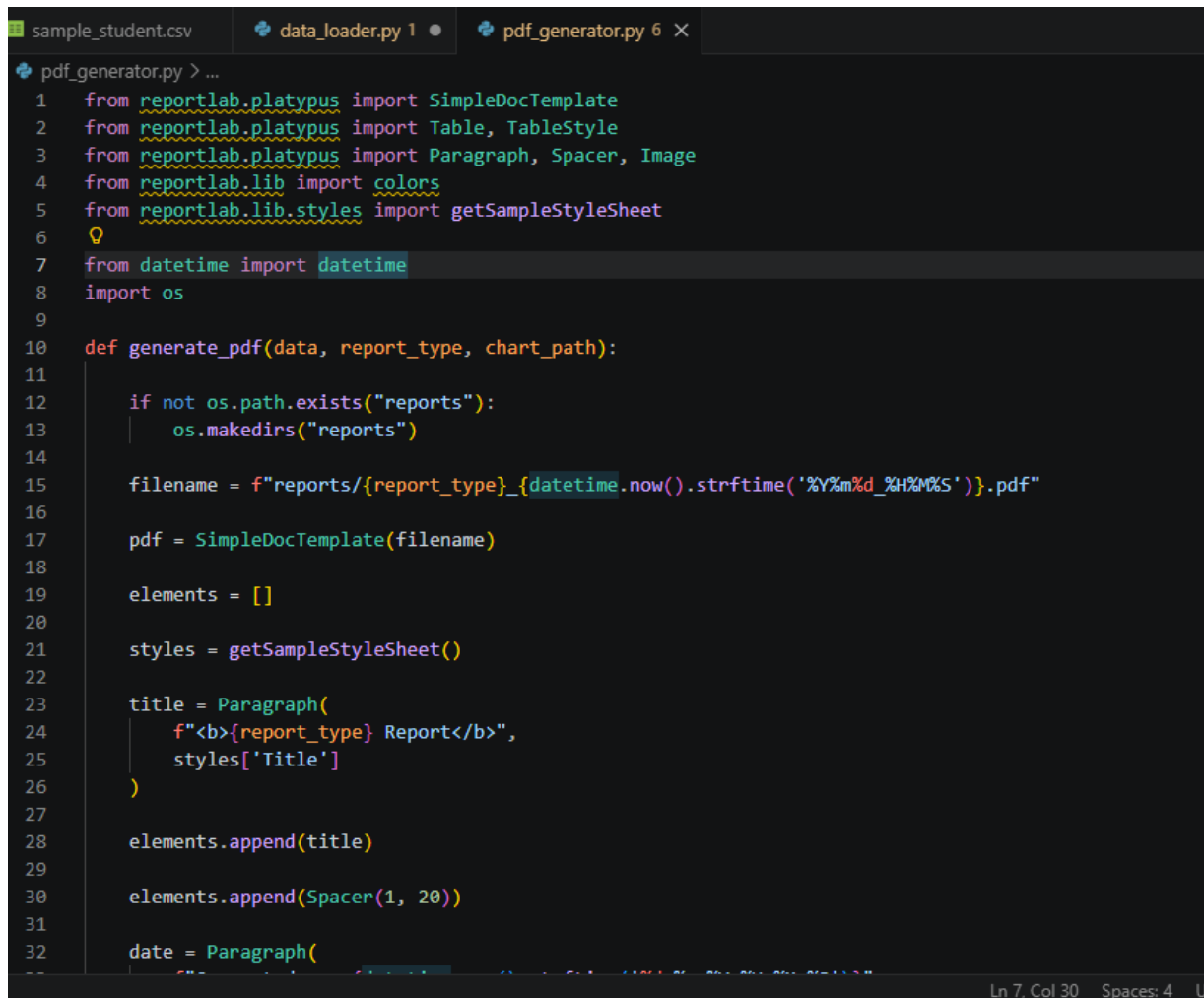
```
sample_student.csv  data_loader.py 1  main.py  X
main.py > ...
1  from data_loader import save_student_data, save_company_data, load_csv
2  from pdf_generator import generate_pdf
3  from charts import generate_student_chart, generate_company_chart
4
5  student_data = []
6  company_data = []
7
8  while True:
9
10     print("\n===== PDF REPORT SYSTEM =====")
11     print("1. Add Data")
12     print("2. Load Data")
13     print("3. Generate PDF")
14     print("4. Exit")
15
16     choice = input("Enter Choice: ")
17
18     # =====
19     # ADD DATA
20     # =====
21     if choice == "1":
22
23         print("\n1. Student Data")
24         print("2. Company Data")
25
26         type_choice = input("Select Type: ")
27
28         # STUDENT
29         if type_choice == "1":
30
31             name = input("Name: ")
32             sid = input("ID: ")
33             save_student_data(name, sid)
```

```
main.py > ...
30
31     name = input("Name: ")
32     sid = input("ID: ")
33     email = input("Email: ")
34     course = input("Course: ")
35     marks = input("Marks: ")
36     attendance = input("Attendance: ")
37
38     row = [name, sid, email, course, marks, attendance]
39
40     student_data.append(row)
41     save_student_data(row)
42
43     print("✅ Student Data Added & Saved!")
44
45     # COMPANY
46     elif type_choice == "2":
47
48         name = input("Name: ")
49         cid = input("ID: ")
50         email = input("Email: ")
51         dept = input("Department: ")
52         role = input("Role: ")
53         performance = input("Performance: ")
54
55         row = [name, cid, email, dept, role, performance]
56
57         company_data.append(row)
58         save_company_data(row)
59
60         print("✅ Company Data Added & Saved!")
61
```

```
60     print("✅ Company Data Added & Saved!")
61
62     else:
63         print("Invalid Type!")
64
65     # =====
66     # LOAD DATA
67     # =====
68     elif choice == "2":
69
70         print("\n1. Load Student Data")
71         print("2. Load Company Data")
72
73         load_choice = input("Select: ")
74
75         if load_choice == "1":
76
77             student_data = load_csv("sample_student.csv")
78             print("✅ Student Data Loaded")
79
80         elif load_choice == "2":
81
82             company_data = load_csv("sample_company.csv")
83             print("✅ Company Data Loaded")
84
85         else:
86             print("Invalid Option")
87
88     # =====
89     # GENERATE PDF
90     # =====
91     elif choice == "3":
```

```
main.py > ...
90 # =====
91 elif choice == "3":
92
93     print("\n1. Student Report")
94     print("2. Company Report")
95
96     report_choice = input("Select Report: ")
97
98     # STUDENT REPORT
99     if report_choice == "1":
100
101         student_data = load_csv("sample_student.csv")
102
103         if len(student_data) == 0:
104             print("No Student Data Found")
105         else:
106             chart = generate_student_chart(student_data)
107             generate_pdf(student_data, "Student", chart)
108
109     # COMPANY REPORT
110     elif report_choice == "2":
111
112         company_data = load_csv("sample_company.csv")
113
114         if len(company_data) == 0:
115             print("No Company Data Found")
116         else:
117             chart = generate_company_chart(company_data)
118             generate_pdf(company_data, "Company", chart)
119
120     else:
121         print("Invalid Option")
122
```

```
118         generate_pdf(company_data, "Company", chart)
119
120     else:
121         print("Invalid Option")
122
123     # =====
124     # EXIT
125     # =====
126     elif choice == "4":
127
128         print("Program Closed 🚪")
129         break
130
131     else:
132         print("Invalid Choice!")
```

Pdf_generator.py:

```
sample_student.csv  data_loader.py 1  pdf_generator.py 6 X
pdf_generator.py > ...
1  from reportlab.platypus import SimpleDocTemplate
2  from reportlab.platypus import Table, TableStyle
3  from reportlab.platypus import Paragraph, Spacer, Image
4  from reportlab.lib import colors
5  from reportlab.lib.styles import getSampleStyleSheet
6
7  from datetime import datetime
8  import os
9
10 def generate_pdf(data, report_type, chart_path):
11
12     if not os.path.exists("reports"):
13         os.makedirs("reports")
14
15     filename = f"reports/{report_type}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.pdf"
16
17     pdf = SimpleDocTemplate(filename)
18
19     elements = []
20
21     styles = getSampleStyleSheet()
22
23     title = Paragraph(
24         f"<b>{report_type} Report</b>",
25         styles['Title']
26     )
27
28     elements.append(title)
29
30     elements.append(Spacer(1, 20))
31
32     date = Paragraph(
33         f"<b>Date: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}</b>"
34     )
35
36     elements.append(date)
37
38     pdf.build(elements)
```

Ln 7, Col 30 Spaces: 4 U

```
10 def generate_pdf(data, report_type, chart_path):
11
12     date = Paragraph(
13         f"Generated on: {datetime.now().strftime('%d-%m-%Y %H:%M:%S')}",
14         styles['Normal']
15     )
16
17     elements.append(date)
18
19     elements.append(Spacer(1, 20))
20
21     if report_type == "Student":
22
23         table_data = [
24             [
25                 "Name",
26                 "ID",
27                 "Email",
28                 "Course",
29                 "Marks",
30                 "Attendance"
31             ]
32         ]
33
34     else:
35
36         table_data = [
37             [
38                 "Name",
39                 "ID",
40                 "Email",
41                 "Department",
42                 "Role",
```

```

10 def generate_pdf(data, report_type, chart_path):
11     email,
12     "Department",
13     "Role",
14     "Performance"
15 ]
16 ]
17
18 for row in data:
19     table_data.append(row)
20
21 table = Table(table_data)
22
23 style = TableStyle([
24     ('BACKGROUND', (0, 0), (-1, 0), colors.grey),
25     ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
26     ('ALIGN', (0, 0), (-1, -1), 'CENTER'),
27     ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
28     ('BOTTOMPADDING', (0, 0), (-1, 0), 12),
29     ('BACKGROUND', (0, 1), (-1, -1), colors.beige),
30     ('GRID', (0, 0), (-1, -1), 1, colors.black),
31 ])
32
33 table.setStyle(style)
34

```

```

    ]
)

table.setStyle(style)

elements.append(table)

elements.append(Spacer(1, 30))

chart = Image(chart_path, width=400, height=250)

elements.append(chart)

pdf.build(elements)

print("PDF Generated Successfully!")
print("Saved As:", filename)

```

OUTPUT;

```
PDF Report Generator for Student Company Data Task

PROBLEMS 8 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS E:\Hasnain Karimain (HK) Educational Academy, Software House & Training> python main.py

===== PDF REPORT SYSTEM =====
1. Add Data
2. Load Data
3. Generate PDF
4. Exit
Enter Choice: 1

1. Student Data
2. Company Data
Select Type: 1
Name: Hashir Buriro
ID: 12
Email: buriro.hashir@gmail.com
Course: Computer
Marks: 86
Attendance: 75
✅ Student Data Added & Saved!

===== PDF REPORT SYSTEM =====
1. Add Data
2. Load Data
3. Generate PDF
4. Exit
Enter Choice: 1

1. Student Data
2. Company Data
Select Type: 2
Name: Engr Hashir Buriro
ID: 123
Email: buriro.hashir@uqconsultants.com
Department: IT
Role: Python Developer
Performance: Excellent
```

Role: Python Developer

Performance: Excellent

✔ Company Data Added & Saved!

===== PDF REPORT SYSTEM =====

1. Add Data
2. Load Data
3. Generate PDF
4. Exit

Enter Choice: 2

1. Load Student Data
2. Load Company Data

Select: 1

✔ Student Data Loaded

===== PDF REPORT SYSTEM =====

1. Add Data
2. Load Data
3. Generate PDF
4. Exit

Enter Choice: 2

1. Load Student Data
2. Load Company Data

Select: 2

Error Loading CSV: Error tokenizing data. C error: Expected 6 fields in line 4, saw 11

✔ Company Data Loaded

===== PDF REPORT SYSTEM =====

1. Add Data
2. Load Data
3. Generate PDF
4. Exit

Enter Choice: 2

```
3. Generate PDF
4. Exit
Enter Choice: 2

1. Load Student Data
2. Load Company Data
Select: 2
✔ Company Data Loaded

===== PDF REPORT SYSTEM =====
1. Add Data
2. Load Data
3. Generate PDF
4. Exit
Enter Choice: 3

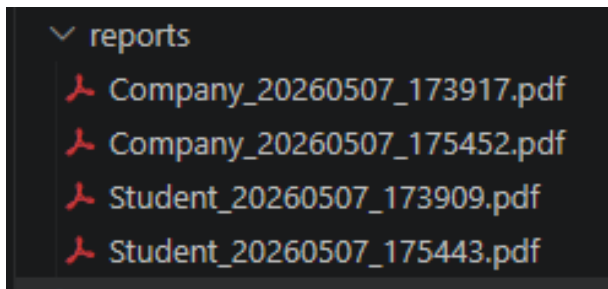
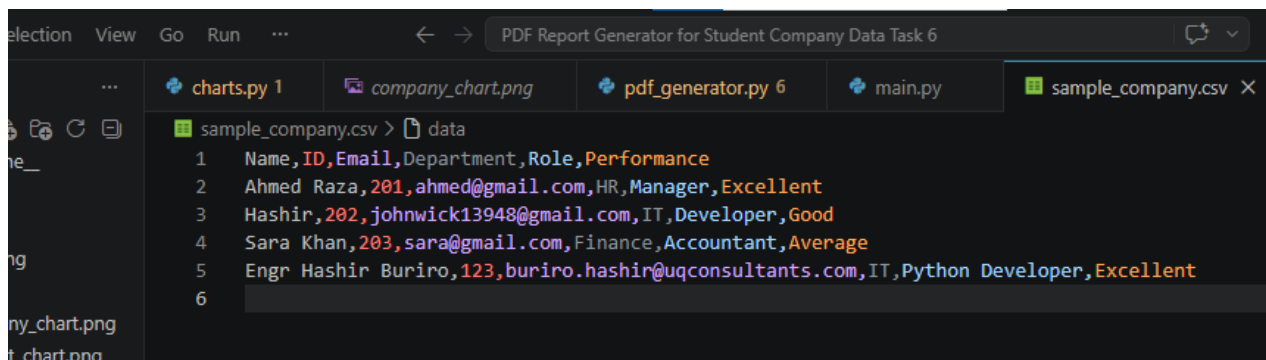
1. Student Report
2. Company Report
Select Report: 1
PDF Generated Successfully!
Saved As: reports/Student_20260507_175443.pdf

===== PDF REPORT SYSTEM =====
1. Add Data
2. Load Data
3. Generate PDF
4. Exit
Enter Choice: 3

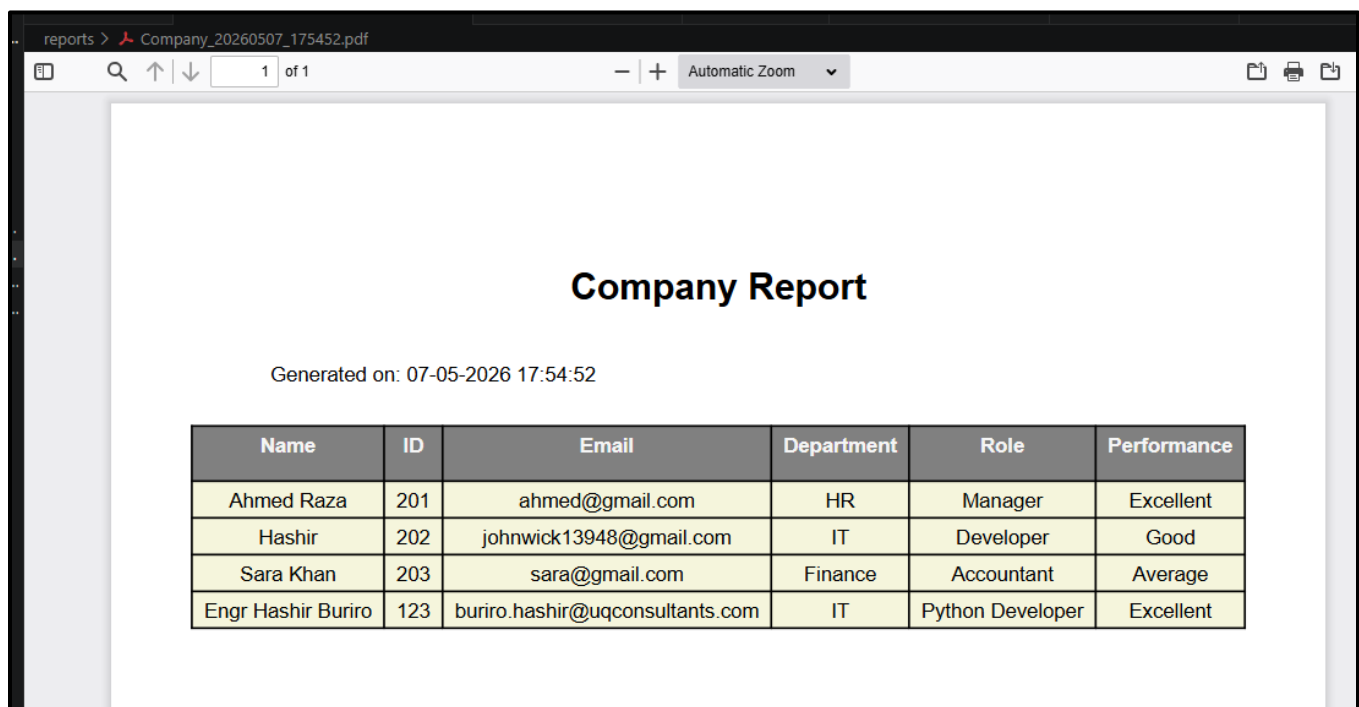
1. Student Report
2. Company Report
Select Report: 2
PDF Generated Successfully!
Saved As: reports/Company_20260507_175452.pdf

===== PDF REPORT SYSTEM =====
1. Add Data
```

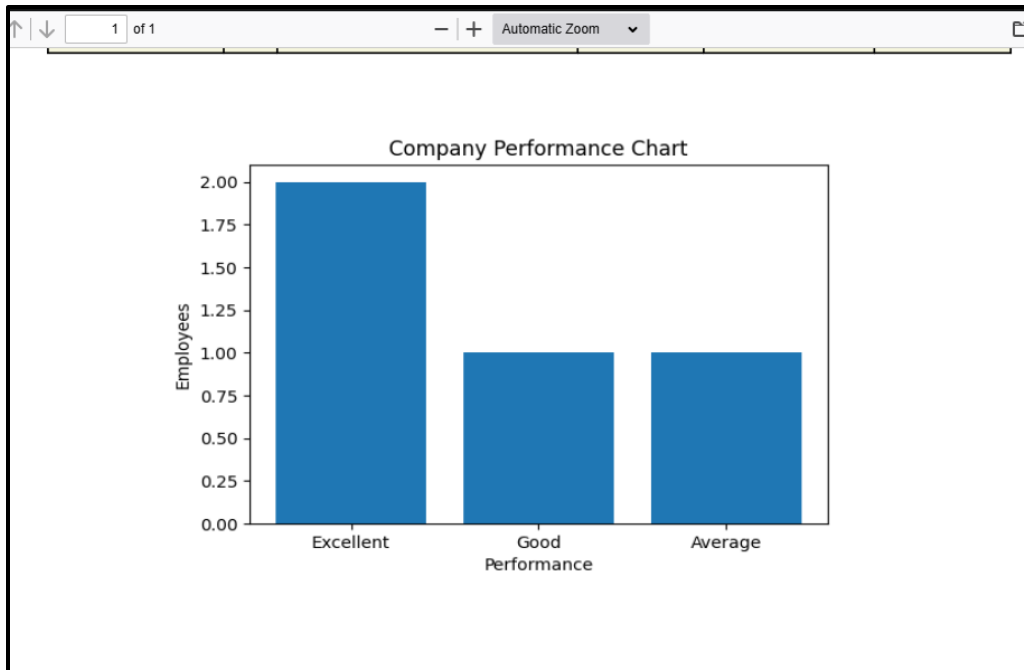
```
===== PDF REPORT SYSTEM =====
1. Add Data
2. Load Data
3. Generate PDF
4. Exit
Enter Choice: 4
Program Closed 🚀
PS E:\Hasnain Karimain (HK) Educational Academy, Software House & Training
```

PDF Generated Successfully:**Company PDF:****Sample_company.csv (Updated Data):**

```
1 Name,ID,Email,Department,Role,Performance
2 Ahmed Raza,201,ahmed@gmail.com,HR,Manager,Excellent
3 Hashir,202,johnwick13948@gmail.com,IT,Developer,Good
4 Sara Khan,203,sara@gmail.com,Finance,Accountant,Average
5 Engr Hashir Buriro,123,buriro.hashir@uqconsultants.com,IT,Python Developer,Excellent
6
```



Company Report					
Generated on: 07-05-2026 17:54:52					
Name	ID	Email	Department	Role	Performance
Ahmed Raza	201	ahmed@gmail.com	HR	Manager	Excellent
Hashir	202	johnwick13948@gmail.com	IT	Developer	Good
Sara Khan	203	sara@gmail.com	Finance	Accountant	Average
Engr Hashir Buriro	123	buriro.hashir@uqconsultants.com	IT	Python Developer	Excellent

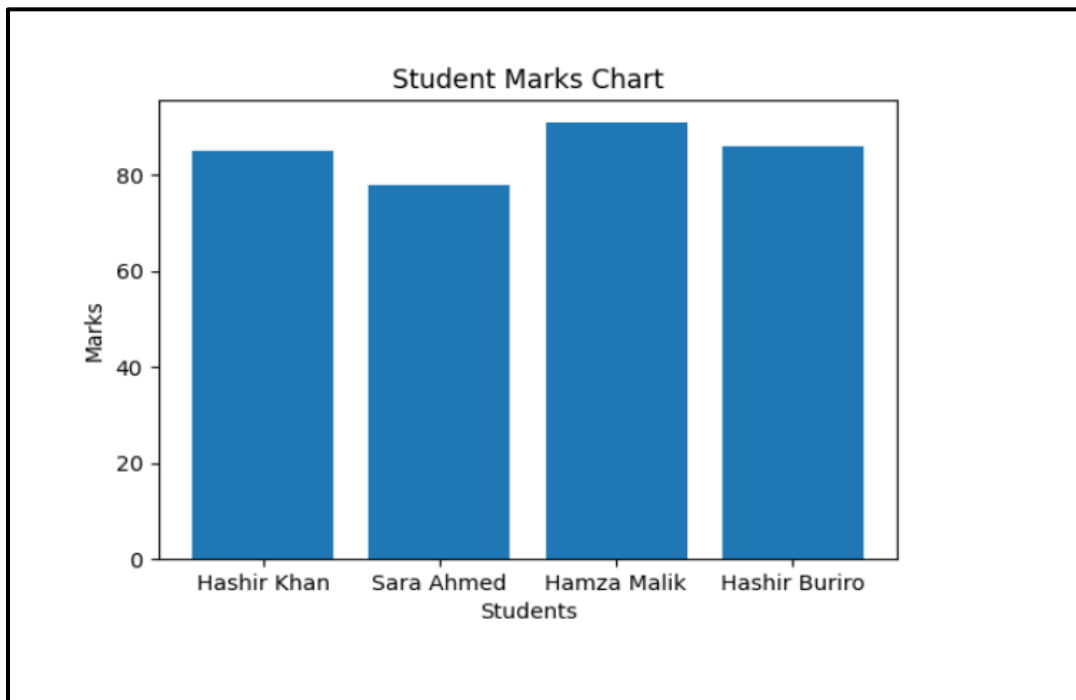
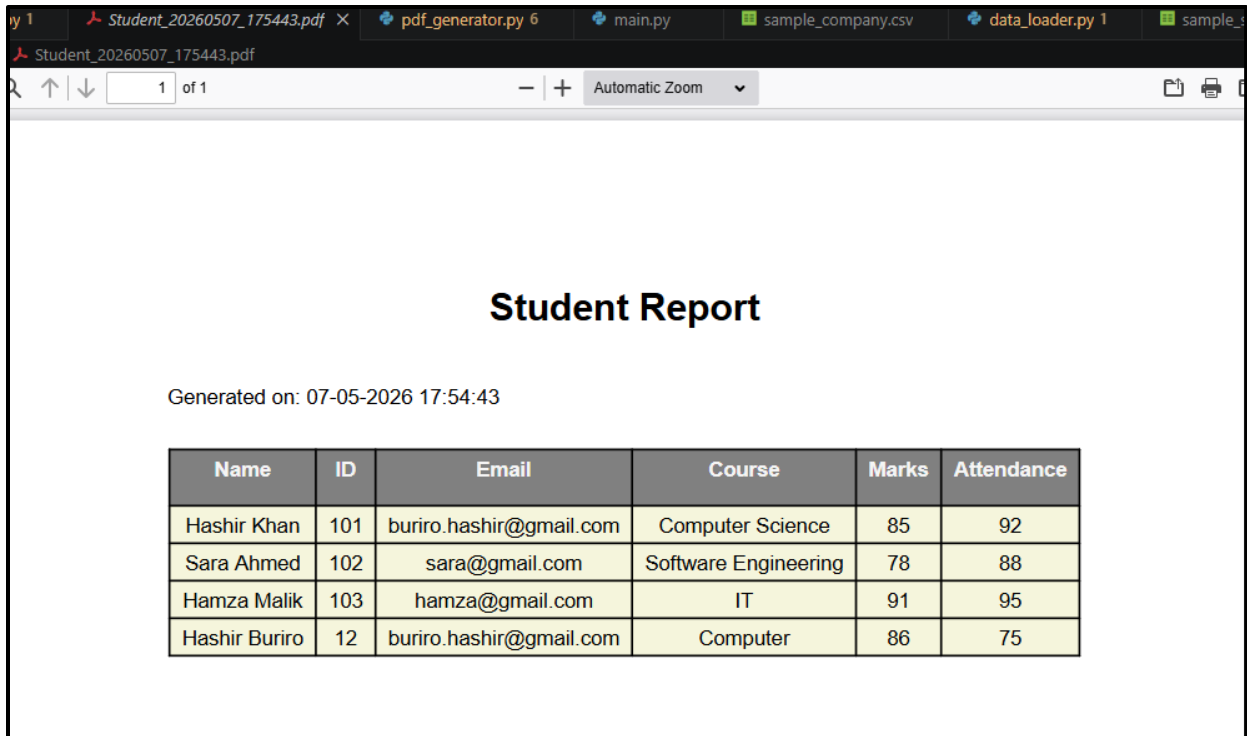


Student Update Data PDF:

Sample_student.csv (Updated Data):

```
PDF Report Generator for Student Company Data Task 6

sample_student.csv X
sample_student.csv > data
1 Name,ID,Email,Course,Marks,Attendance
2 Hashir Khan,101,buriro.hashir@gmail.com,Computer Science,85,92
3 Sara Ahmed,102,sara@gmail.com,Software Engineering,78,88
4 Hamza Malik,103,hamza@gmail.com,IT,91,95
5 Hashir Buriro,12,buriro.hashir@gmail.com,Computer,86,75
6
```



Project Result

After successfully running this project, the system works as a complete **dynamic PDF Report Generator** for both student and company data. Whenever the user adds new data through the “Add Data” option, it is automatically saved into the CSV file and updated in real time. The “Load Data” option retrieves the latest data from the CSV file, while the “Generate PDF” option converts this updated data into a professionally formatted report. The student report displays a structured marksheet including marks and attendance, while the company report shows employee details such as department, role, performance, along with graphical visualization. Each generated PDF is saved in the reports folder with a unique filename and always reflects the latest updated data.

Conclusion

This project demonstrates the practical implementation of Python in building a real-world reporting system using libraries like Report Lab, Pandas, and Matplotlib. It combines data input, file handling, data persistence, and automated PDF generation into a single application. The system ensures that all data remains updated and dynamically reflected in reports without manual intervention. This project strengthens core programming concepts such as modular coding, file management, and data visualization, and it can be further extended for real-world use cases like academic mark sheets, HR employee reports, and business performance dashboards.